

PYTHON REFLEX BOOK

600 questions to make Python your first instinct

Core Python only | No solutions | Topics in [brackets] | Easy to puzzle-level

Solve. Break. Rewrite. Repeat.

How to use this book

- Pace: 5 questions a day = 120 days for the whole book. Consistency beats speed.
- Har question pe pehle 25 minute khud try karo. Solution dhoondhne se pehle 2 din ruko, dimaag ko struggle karne do.
- Pehle **brute force / ugly** version likho jo chale. Phir usko Pythonic banao. Dono versions save karo, gap hi seekhna hai.
- Har question ke liye khud edge cases likho: empty, ek item, duplicates, negative, bahut bada. **assert** se test karo.
- Brackets [...] batate hain kaunse tools relevant hain. Woh approach ka hint nahi hain.
- **Bug log** rakho: har galti ki ek line. Har hafte padho. Wahi tumhari asli weak spots hain.
- **Reflex loop**: jo question atka ya lookup karna pada, usse Day+3, Day+10, Day+30 pe bina dekhe dobara solve karo.
- **Blank-screen test** (har weekend): 5 purane random questions, koi notes nahi, 30 minute. Yehi reflex banata hai.

Rules of the game

- Core Python only: lists, tuples, dicts, sets, strings, loops, functions, comprehensions, classes, files.
- Allowed imports: **math**, **random**, **sys**, **time**, **json**, **abc**, **heapq**, **functools**, and **collections.deque**. Anything else, write it by hand.
- Where a question says 'by hand' or 'without X', obey it. The restriction is the lesson.
- After solving, ask three things: Can it be shorter? Can it be faster? What input breaks it?

Levels

Level	Feel	How to treat it
2 to 3	One idea, builds reflex	Should take under 10 minutes. If not, revisit the basics.
4 to 6	One 'aha' needed	Expect to get stuck once. Sleep on it before looking anything up.
7 to 8	Two ideas combined, or nasty edge cases	Break into sub-problems. Test on paper first.
9	Puzzle	You will get stuck. That is the point. Come back after a week.

Contents

#	Chapter	Questions	Level 2-3 / 4-6 / 7-8 / 9
1	Names, Objects & Types	Q1 to Q30	10 / 14 / 5 / 1
2	Numbers & Math Thinking	Q31 to Q60	10 / 14 / 5 / 1
3	Strings I: Slicing, Scanning & Classic Problems	Q61 to Q90	13 / 13 / 3 / 1
4	Strings II: Parsing, Formatting & Text Processing	Q91 to Q120	9 / 17 / 3 / 1
5	Lists: Manipulation, Patterns & Techniques	Q121 to Q150	10 / 16 / 3 / 1
6	Tuples, Unpacking & Data Modelling	Q151 to Q180	10 / 15 / 4 / 1
7	Dictionaries: Counting, Grouping, Lookup & Caching	Q181 to Q210	11 / 16 / 2 / 1
8	Sets: Uniqueness, Membership & Set Thinking	Q211 to Q240	10 / 16 / 3 / 1
9	Loops, Conditions & Simulation	Q241 to Q270	11 / 16 / 2 / 1
10	Functions: Design, Scope, Closures & Decorators	Q271 to Q300	10 / 16 / 3 / 1
11	Comprehensions, Generators & the Iteration Protocol	Q301 to Q330	10 / 16 / 3 / 1
12	Sorting & Searching: Keys, Stability & Binary Search Thinking	Q331 to Q360	9 / 17 / 3 / 1
13	Recursion & Backtracking	Q361 to Q390	10 / 15 / 4 / 1
14	Classes & Object-Oriented Design	Q391 to Q420	10 / 16 / 3 / 1
15	Exceptions & File Handling	Q421 to Q450	10 / 16 / 3 / 1
16	Grids & Matrices	Q451 to Q480	10 / 16 / 3 / 1
17	Two Pointers, Windows, Prefix Sums & Greedy	Q481 to Q510	8 / 18 / 3 / 1
18	Stacks, Queues, Linked Structures & Heaps	Q511 to Q540	9 / 17 / 3 / 1
19	Real-World Mini Projects & Data Wrangling	Q541 to Q570	6 / 20 / 3 / 1
20	Debugging, Output Prediction & Python Puzzles	Q571 to Q600	9 / 15 / 4 / 2

CHAPTER 1

Names, Objects & Types

Python ka sabse bada secret: variables "boxes" nahi, "labels" hain. Yeh chapter tumhari mental model theek karega.

Q1 ●●○○○○○○○○ 2/10 [type(), int/float/str/bool, operators]

Predict the type and value of each, before running: `3/2`, `3//2`, `3.0//2`, `-3//2`, `True+True`, `"3"*3`, `[0]*3`, `"a"+str(1)`. Explain every result in one sentence.

Q2 ●●○○○○○○○○ 2/10 [truthiness, filter]

List every kind of value that is falsy in Python (at least 8). Then write one line that takes a list of mixed values and prints only the truthy ones.

Q3 ●●○○○○○○○○ 2/10 [int(), exceptions, defaults]

Write `safe_int(s, default=0)` that returns `int(s)` or the default. Decide what should happen with `" 42 "`, `"4_2"`, `"3.0"`, `"0x1f"`, `" "` and justify each decision by testing.

Q4 ●●○○○○○○○○ 2/10 [assignment, swap]

Swap two variables in three different ways: with a temp, with tuple unpacking, and with arithmetic. Why does the arithmetic way fail for strings and can it overflow in Python?

Q5 ●●●○○○○○○○○ 3/10 [is vs ==, small ints]

`a = 256; b = 256; c = 257; d = 257`. Predict `a is b` and `c is d` in a script and in the REPL. Why must you never rely on this?

Q6 ●●●○○○○○○○○ 3/10 [aliasing, mutability]

`a = [1,2,3]; b = a; b.append(4)`. What is `a`? Now repeat with a tuple and a string using `+=`. Explain why the results differ.

Q7 ●●●○○○○○○○○ 3/10 [id(), mutation vs rebinding]

Write three functions that "add x to a list": one using `lst += [x]`, one using `lst = lst + [x]`, one using `lst.append(x)`. Predict which ones change the caller's list, then verify with `id()`.

Q8 ●●●○○○○○○○○ 3/10 [default arguments]

Predict the output, then fix the function properly:

```
def f(x, acc=[]):
    acc.append(x)
    return acc
print(f(1)); print(f(2)); print(f(3, []))
```

Q9 ●●●○○○○○○○○ 3/10 [copying, nested lists]

Create a 3x3 grid with `[[0]*3]*3`, set `grid[0][0] = 1` and print it. Explain the bug and give three different correct ways to build the grid.

Q10 ●●●○○○○○○ 3/10 [bool is int, arithmetic]

Count how many numbers in a list are even without an `if` statement and without a loop body, using the fact that `True == 1`.

Q11 ●●●●○○○○○○ 4/10 [is None, __eq__]

Why is `x is None` preferred over `x == None`? Build a small class whose `==` always returns `True` and show that `is` still tells the truth.

Q12 ●●●●○○○○○○ 4/10 [recursion on structures, copying]

Without importing anything, write `deep_copy(x)` for arbitrarily nested lists, dicts and tuples of ints and strings. Show it differs from `x[:]` and `dict(x)`.

Q13 ●●●●○○○○○○ 4/10 [short-circuit, or/and]

Predict: `0 or "" or [] or "hi", 1 and 2 and 0 and 3, None or 0, [] and 5`. State the rule in one sentence, then write a one-line `first_non_empty(a, b, c)`.

Q14 ●●●●○○○○○○ 4/10 [chained comparison]

Predict: `1 < 2 < 3, 3 > 2 > 2, 1 == 1 == 1, 1 < 2 > 1, 1 < 3 < 2`. Then write `is_between(x, lo, hi)` without using `and`.

Q15 ●●●●○○○○○○ 4/10 [float precision]

Why does `0.1 + 0.2 == 0.3` fail? Write `almost_equal(a, b)` and explain why a fixed absolute tolerance is wrong for very large or very small numbers.

Q16 ●●●●●○○○○○ 5/10 [big ints, digits]

Find the digit sum of `2**1000`. Then find the number of trailing zeros of `100!` two ways: by brute force and by a formula. Why does Python not overflow here?

Q17 ●●●●●○○○○○ 5/10 [isinstance, bool trap]

Write `describe(x)` returning 'bool', 'int', 'float', 'str', 'list' or 'other'. Why does the order of your checks matter? What goes wrong with `isinstance(True, int)`?

Q18 ●●●●●○○○○○ 5/10 [tuple mutability, augmented assignment]

`t = (1, [2, 3])`. Predict what `t[1].append(4)` does, then what `t[1] += [5]` does (it raises AND modifies). Explain step by step what happens.

Q19 ●●●●●○○○○○ 5/10 [closures, late binding]

`funcs = [lambda: i for i in range(3)]`. Predict `[f() for f in funcs]`. Fix it in two different ways.

Q20 ●●●●●○○○○○ 5/10 [scope, global, nonlocal]

Why does `x = 1` followed by `def f(): print(x); x = 2` raise `UnboundLocalError`? Write three counters: with `global`, with a list, and with a closure using `nonlocal`.

Q21 ●●●●●○○○○ 6/10 [hashing, immutability]

Why can a tuple be a dict key but a list cannot? Make a tuple that still fails as a key. Then write `freeze(x)` that converts nested lists/dicts/sets into hashable equivalents.

Q22 ●●●●●○○○○ 6/10 [strings immutable, performance]

Build a string of 100,000 characters (a) with `+=` in a loop and (b) with `join`. Time both with `time.time()`. What does string immutability imply, and why is `+=` sometimes fast anyway?

Q23 ●●●●●○○○○ 6/10 [references inside containers]

`a = [1, 2]; b = [a, a]; b[0].append(3)`. Predict `b`. Then create a list that contains itself, print it, and explain what Python prints and why it does not loop forever.

Q24 ●●●●●○○○○ 6/10 [augmented assignment, `__iadd__`]

Predict: `a = [1,2]; b = a; a += [3]; print(b)` and then `a = a + [4]; print(b)`. Explain `__iadd__` versus `__add__` in plain words.

Q25 ●●●●●○○○○ 7/10 [sentinel objects, default args]

Write `f(x, default=None)` where `None` is a legitimate value a caller might pass. How do you tell "argument not passed" from "passed None"?

Q26 ●●●●●○○○○ 7/10 [chained assignment]

Compare `a = b = []` with `a, b = [], []`. Then predict `x = y = 5; x += 1; print(y)`. Describe a realistic bug where the first form hurts you.

Q27 ●●●●●○○○○ 7/10 [reference counts, `del`]

Using `sys.getrefcount`, show how an object's count changes as you add names, put it inside a list, and `del` names. Why is the printed number one higher than you expect?

Q28 ●●●●●○○○○ 7/10 [NaN, identity shortcut in containers]

`nan = float('nan')`. Predict `nan == nan`, `nan in [nan]`, `[nan] == [nan]`. Explain each result, especially the container ones.

Q29 ●●●●●○○○ 8/10 [class attributes, default evaluation time]

Write `class Team` with `__init__(self, members=[])` and show two teams sharing members. Prove that the default is created once at definition time, using a default like `time.time()`. Design a clean fix.

Q30 ●●●●●○○○ 9/10 [everything: names, objects, mutation]

Predict the exact output without running:

```
a = [1, 2, 3]
b = a
c = a[:]
a.append(a)
b = b + [9]
c += a
print(len(a), len(b), len(c))
print(a is a[3], c[3] is a)
```

CHAPTER 2

Numbers & Math Thinking

Loops aur arithmetic se sochna seekho. Yahan formulas se pehle observation aur tests aane chahiye.

Q31 ●●○○○○○○○○ 2/10 [%, //, divmod, formatting]

Convert a number of seconds into an `H:MM:SS` string without imports. For example 3725 becomes `1:02:05`. Test on 0, 59, 60, 3599, 86400.

Q32 ●●○○○○○○○○ 2/10 [%, loops, conditions]

Print all numbers from 1 to 100 that are divisible by 3 or 5 but not by both.

Q33 ●●○○○○○○○○ 2/10 [digits, //, %]

Sum the digits of an integer using arithmetic only (no `str`). Then do it with `str` and compare speed on a 5000-digit number.

Q34 ●●○○○○○○○○ 2/10 [range with step, join]

Print the multiplication table of `n` on a single line separated by `' | '`, using `join`.

Q35 ●●○○○○○○○○ 2/10 [min, max, arithmetic]

Find the median of three numbers using only `min`, `max` and arithmetic, no sorting and no `if`.

Q36 ●●●○○○○○○○○ 3/10 [round, banker's rounding]

Predict `round(0.5)`, `round(1.5)`, `round(2.5)`, `round(-0.5)`, `round(2.675, 2)`. Write `round_half_up(x)` that behaves the way school taught you.

Q37 ●●●○○○○○○○○ 3/10 [digits, sign handling]

Reverse the digits of an integer keeping its sign (-1230 becomes -321). Do it with strings and with arithmetic.

Q38 ●●●○○○○○○○○ 3/10 [palindromes, digits]

Check whether an integer is a palindrome without converting it to a string.

Q39 ●●●○○○○○○○○ 3/10 [Euclid, while loop]

Write `gcd` with Euclid's algorithm using a `while` loop, then `lcm` using it. Explain why Euclid works in one sentence.

Q40 ●●●○○○○○○○○ 3/10 [primes, edge cases]

Write `is_prime(n)` that is correct for 0, 1, 2, negative numbers and perfect squares of primes. Test on the first 50 integers.

Q41 ●●●●○○○○○○○○ 4/10 [sieve, slicing]

List all primes below 10,000 with the Sieve of Eratosthenes. Why is starting the inner loop at `p*p` correct? Can you use slice assignment to speed it up?

Q42 ●●●●○○○○○ 4/10 [factorization, dict]

`factorize(360)` should give `{2: 3, 3: 2, 5: 1}`. Use it to count the divisors of any number and to compute the sum of its divisors.

Q43 ●●●●○○○○○ 4/10 [bases, divmod, strings]

Convert a non-negative int to any base 2 to 16 as a string, and back, without `bin`, `hex` or `int(s, base)`.

Q44 ●●●●○○○○○ 4/10 [Armstrong numbers, generalization]

Find all Armstrong numbers up to 100,000 ($153 = 1^3 + 5^3 + 3^3$). Generalize your code to any number of digits.

Q45 ●●●●○○○○○ 4/10 [carry, strings of digits]

Add two numbers given as digit strings ('999' and '1') without converting the whole strings to `int`. Then subtract, assuming the first is larger.

Q46 ●●●●○○○○○ 5/10 [Collatz, memoization]

For starting numbers 1 to 100,000 find the one with the longest Collatz chain. Speed it up by remembering chain lengths you already computed.

Q47 ●●●●○○○○○ 5/10 [binary search, integer sqrt]

Implement `isqrt(n)` without `**0.5` and without `math`, using binary search. Why is `int(n**0.5)` unsafe for huge `n`?

Q48 ●●●●○○○○○ 5/10 [repeated squaring]

Compute `a**b % m` for a huge `b` (say 10^{18}) with repeated squaring. Compare with the three-argument `pow`. How many multiplications did you need?

Q49 ●●●●○○○○○ 5/10 [tuples as fractions, invariants]

Represent fractions as `(num, den)` tuples. Write add, multiply, compare and reduce. Enforce a positive denominator. What should happen when `den` is 0?

Q50 ●●●●○○○○○ 5/10 [long division, dict of remainders]

Compute the first 50 digits of $1/7$, and for any `n` detect the length of the repeating cycle of $1/n$ using long division and a dict of seen remainders.

Q51 ●●●●●○○○○ 6/10 [Roman numerals, validation]

Convert int to Roman and Roman to int (1 to 3999). Make the reverse direction reject `'IIII'`, `'VX'` and `'IC'`.

Q52 ●●●●●○○○○ 6/10 [number to words, decomposition]

Convert 0 to 999,999 to English words: 1042 becomes `'one thousand forty-two'`. Cover 0, 10 to 19, and round numbers.

Q53 ●●●●●○○○○ 6/10 [odd-number sums, no floats]

Check whether n is a perfect square using only addition (the sum $1 + 3 + 5 + \dots$ trick). Prove to yourself why it works.

Q54 ●●●●●○○○○ 6/10 [digital root, patterns]

Compute the digital root of n (repeat the digit sum until one digit remains). Then find a one-line formula using `% 9`, and verify it against brute force for 1 to 10,000.

Q55 ●●●●●○○○○ 7/10 [cycles, brute force over inputs]

Take a 4-digit number (not all digits equal), sort digits descending minus ascending, repeat. Show it always reaches 6174 and find the maximum number of steps over all valid inputs.

Q56 ●●●●●○○○○ 7/10 [bit operations]

Without `bin()`, count the set bits of n . Check for a power of two in one expression. Find the single number that appears once in a list where every other appears twice (XOR trick).

Q57 ●●●●●○○○○ 7/10 [extended Euclid, modular inverse]

Write extended Euclid returning (g, x, y) with $a*x + b*y = g$ and use it to find the modular inverse of a modulo m . When does the inverse not exist?

Q58 ●●●●●○○○○ 7/10 [divisor sums, efficiency]

Find all amicable pairs below 10,000. Make the divisor-sum computation fast enough to run in under a second.

Q59 ●●●●●○○○○ 8/10 [digit lists, grade-school algorithm]

Multiply two 50-digit numbers stored as lists of digits without converting either to `int`. Verify against Python's own multiplication with random tests.

Q60 ●●●●●○○○○ 9/10 [reasoning, verification by brute force]

Find the last non-zero digit of $100!$. First reason it out with counting factors of 2 and 5, then verify with brute force. Then generalize to $n!$ for n up to 10^5 .

CHAPTER 3

Strings I: Slicing, Scanning & Classic Problems

Strings me 60% problems ek hi pattern ke variations hote hain: scan, count, compare. Pattern pakdo.

Q61 ●●○○○○○○○○ 2/10 [slicing, split, join, reversing]

Reverse each word in a sentence but keep the word order. Then reverse the word order but keep each word intact.

Q62 ●●○○○○○○○○ 2/10 [counting, lower, membership]

Count vowels in a string, case-insensitively. Then count consonants ignoring digits, spaces and punctuation.

Q63 ●●○○○○○○○○ 2/10 [strip, split, join]

Normalise ' Hello, World ! ' to 'Hello, World !': collapse repeated spaces and trim the ends, using no loops.

Q64 ●●○○○○○○○○ 2/10 [find vs index, overlapping matches]

What is the difference between `find` and `index` when the substring is missing? Write `count_overlapping(s, sub)` (counting 'aa' in 'aaaa' gives 3).

Q65 ●●○○○○○○○○ 2/10 [case conversion, string methods]

Convert `snake_case` to `camelCase` and `camelCase` to `snake_case`. Handle a leading underscore and consecutive capitals such as `HTTPServer`.

Q66 ●●○○○○○○○○ 2/10 [ord, chr, modulo]

Write a Caesar cipher with any shift (including negative and larger than 26) that preserves case and leaves non-letters alone. Then write a function that breaks it by trying all shifts.

Q67 ●●●○○○○○○○○ 3/10 [str.is* methods, collecting errors]

Validate a password: at least 8 characters, one upper, one lower, one digit, one special character, no spaces. Return the list of ALL rules it breaks, not only the first.

Q68 ●●●○○○○○○○○ 3/10 [palindromes, cleaning]

Check for a palindrome ignoring case, spaces and punctuation. Then find the longest palindromic substring by brute force.

Q69 ●●●○○○○○○○○ 3/10 [anagrams, three counting methods]

Are two strings anagrams? Do it three ways: sorting, counting with a dict, and counting with a fixed 26-element list.

Q70 ●●●○○○○○○○ 3/10 [first unique, counting]

Return the index of the first non-repeating character of a string, or -1. Make it work in two passes and explain why one pass is not enough.

Q71 ●●●○○○○○○○ 3/10 [versions, tuple comparison]

Compare version strings like '1.10.2' and '1.9' correctly (return -1, 0 or 1). Why does plain string comparison give the wrong answer?

Q72 ●●●○○○○○○○ 3/10 [slicing with steps]

Given a string and k, return every k-th character for every starting offset: 'abcdefgh', k=3 gives ['adg', 'beh', 'cf'].

Q73 ●●●○○○○○○○ 3/10 [dict, sorted, grouping]

Group words that are anagrams of each other. Return the groups sorted by size (largest first), ties by the alphabetically smallest word; words inside a group are sorted too. Input: ["listen", "silent", "enlist", "google", "gogole", "cat", "act", "tac", "dog"].

Q74 ●●●●○○○○○○○ 4/10 [run-length encoding]

Compress 'aaabccddd' to 'a3b1c2d4' and decompress it. What breaks if the original text contains digits? Design an encoding that survives them.

Q75 ●●●●○○○○○○○ 4/10 [word frequency, sorting keys]

Word frequency of a paragraph, ignoring case and punctuation. Print the top 5, ties broken alphabetically. Handle apostrophes in words like "don't".

Q76 ●●●●○○○○○○○ 4/10 [simultaneous replacement]

Replace {'cat': 'dog', 'dog': 'cat'} in 'cat and dog' to get 'dog and cat'. Why does calling replace twice fail? Solve it for arbitrary replacement dicts.

Q77 ●●●●○○○○○○○ 4/10 [partition, split with maxsplit]

Parse 'key = value = more' into ('key', 'value = more') in three different ways.

Q78 ●●●●○○○○○○○ 4/10 [ordered dedupe]

Remove duplicate characters keeping first occurrences: 'programming' becomes 'progamin'. Then keep only characters that appear exactly once.

Q79 ●●●●○○○○○○○ 4/10 [edit distance of one]

Check whether two strings are exactly one edit apart (insert, delete or replace one character). Handle the length difference cleverly instead of trying all edits.

Q80 ●●●●●○○○○○○○ 5/10 [isomorphic strings, bijection]

Two strings are isomorphic if characters of one can be consistently replaced to get the other ('egg'/'add' yes, 'foo'/'bar' no, 'ab'/'aa' no). The mapping must be one-to-one in both directions.

Q81 ●●●●●○○○○○ 5/10 [pattern matching, two dicts]

Given pattern 'abba' and a sentence 'dog cat cat dog', decide whether the words follow the pattern. Test 'abba' against 'dog dog dog dog'.

Q82 ●●●●●○○○○○ 5/10 [rotations, in operator]

Check whether s2 is a rotation of s1 with a single `in` check. Why does it work? Then find the rotation amount.

Q83 ●●●●●○○○○○ 5/10 [longest common prefix]

Longest common prefix of a list of strings, in three ways: character-by-character, using `min/max`, and using `zip`.

Q84 ●●●●●○○○○○ 5/10 [sliding window, dict of last seen]

Length of the longest substring without repeating characters. Return the substring as well.

Q85 ●●●●●○○○○○ 6/10 [big-number strings]

Multiply two non-negative numbers given as strings without using `int()`. Verify with random tests against real multiplication.

Q86 ●●●●●○○○○○ 6/10 [wildcards, recursion]

Implement wildcard matching where `?` matches any single character and `*` matches any sequence (including empty). Write it recursively first, then explain why it explodes and how to fix it.

Q87 ●●●●●●○○○ 7/10 [window counts, two pointers]

Find the smallest window in `s` that contains all characters of `t` (with multiplicity). Return the window or an empty string.

Q88 ●●●●●●○○○ 7/10 [stack, nesting]

Decode strings like '3[a2[c]]' into 'accaccacc' (nested repeats). Do it with recursion and then with an explicit stack.

Q89 ●●●●●●●○○ 8/10 [expand around centers]

Find the longest palindromic substring in $O(n^2)$ by expanding around centers. Why are there exactly $2n - 1$ centers?

Q90 ●●●●●●●○○ 9/10 [dynamic programming, reconstruction]

Compute the minimum number of insert/delete/replace edits to turn one string into another. Print one actual edit sequence, not only the number.

CHAPTER 4

Strings II: Parsing, Formatting & Text Processing

Real-life Python = text ko todna aur jodna. Yahan har question ek chhota tool hai jo tum asli kaam me use kar sakte ho.

Q91 ●●○○○○○○○○ 2/10 [f-strings, alignment]

Print a receipt: item name left-aligned (15 chars), quantity right-aligned (4), price with 2 decimals right-aligned (10), and a total line. Use only f-strings.

Q92 ●●○○○○○○○○ 2/10 [split, empty fields, quotes]

What does `'a,b,,d'.split(',')` give? Then write your own parser that handles quoted fields like `'a,"b,c",d'` correctly.

Q93 ●●○○○○○○○○ 2/10 [format specs]

Print `1234567.891` as `1,234,567.89`, as `1.23e+06`, zero-padded to width 12, and `0.256` as a percentage.

Q94 ●●○○○○○○○○ 2/10 [center, ljust, box drawing]

Center a title in a 40-character banner with `=` padding, then draw a box around any multi-line text.

Q95 ●●●○○○○○○○○ 3/10 [wc, whitespace]

Write `wc(text)` returning lines, words and characters like the Unix `wc`. Decide what a "word" is; handle tabs, multiple spaces and a trailing newline.

Q96 ●●●○○○○○○○○ 3/10 [tokenizing]

Split `'3+4*(2-1)'` into tokens `['3', '+', '4', '*', '(', '2', '-', '1', ')']`, handling multi-digit numbers and decimals like `12.5`.

Q97 ●●●○○○○○○○○ 3/10 [translate, maketrans]

Remove all punctuation from text in one call, without loops or regex. Then write a leetspeak converter (a to 4, e to 3, ...).

Q98 ●●●○○○○○○○○ 3/10 [zfill, validation]

Format a list of (h, m, s) times as HH:MM:SS and parse them back. Reject `'25:61:00'` with a helpful error.

Q99 ●●●○○○○○○○○ 3/10 [format_map, missing keys]

Fill `'Hello {name}, you owe {amount:.2f}'` from a dict using `format_map`. What happens with a missing key? Make a safe default.

Q100 ●●●●○○○○○○ 4/10 [parsing into structures]

Parse `'name=Ali;age=21;skills=py,c,java'` into a nested dict where `skills` is a list and `age` is an int.

Q101 ●●●●○○○○○○ 4/10 [validation without regex]

Validate an email using simple rules and no regex: exactly one @, non-empty local part, domain has a dot that is not at either end, no spaces.

Q102 ●●●●○○○○○○ 4/10 [validation, ranges]

Validate an IPv4 address: four parts, each 0 to 255, no leading zeros unless the part is exactly '0'. Test 15 tricky inputs of your own.

Q103 ●●●●○○○○○○ 4/10 [ASCII art, loops]

Print a diamond of height n, then a right-aligned pyramid, then Pascal's triangle centred.

Q104 ●●●●○○○○○○ 4/10 [text statistics]

Find the longest word, the average word length, and all words that appear exactly once in a paragraph.

Q105 ●●●●○○○○○○ 4/10 [sentences, state]

Capitalize the first letter of every sentence in a paragraph (after ., ! or ?), without breaking abbreviations you decide to support.

Q106 ●●●●○○○○○○ 5/10 [abbreviations]

Convert 'Portable Network Graphics' to 'PNG'. Then, given a list of names, generate a unique 3-letter code for each (resolving clashes deterministically).

Q107 ●●●●○○○○○○ 5/10 [wrapping]

Wrap text to at most W characters per line without breaking words and without `textwrap`. What should happen if a single word is longer than W?

Q108 ●●●●○○○○○○ 5/10 [tables]

Given a list of dicts with the same keys, print a neat table with auto-sized columns and a header underline. Right-align numbers, left-align text.

Q109 ●●●●○○○○○○ 5/10 [markdown-lite]

Convert lines starting with '# ' and '## ' to <h1> and <h2>, and **bold** to bold, without regex. What about unmatched **?**

Q110 ●●●●○○○○○○ 5/10 [encoding, dict lookup]

Encode text to Morse and back with a dict; words are separated by ' / '. Handle unknown characters gracefully.

Q111 ●●●●○○○○○○ 5/10 [boundaries]

Write `human(n)`: 1500 gives '1.5K', 2,300,000 gives '2.3M', 999 gives '999'. Be careful at boundaries like 999,999 and 999,950.

Q112 ●●●●●○○○○ 6/10 [tokenizer and evaluator]

Evaluate `'3 + 4 * 2 - 6 / 3'` respecting operator precedence, without `eval()`. Parentheses come later.

Q113 ●●●●●○○○○ 6/10 [log parsing]

Given lines like `2026-01-05 14:03:22 ERROR db timeout after 30s`, count lines per level and find the most frequent word in ERROR lines.

Q114 ●●●●●○○○○ 6/10 [line diff]

Given two multi-line strings, print the lines that differ as `- old / + new` with line numbers. Decide how to handle inserted lines.

Q115 ●●●●●○○○○ 6/10 [template engine]

Replace `{{ name }}` placeholders in a text using a dict, allowing spaces inside the braces and leaving unknown ones unchanged.

Q116 ●●●●●○○○○ 6/10 [dictionary compression]

Encode a string by replacing each distinct word with its index in a word list, then decode it. Compare sizes on a real paragraph.

Q117 ●●●●●○○○○ 7/10 [recursion, indentation]

Write `pretty(obj)` that prints nested dicts/lists with indentation like `json.dumps(indent=2)`, without importing `json`.

Q118 ●●●●●○○○○ 7/10 [recursive descent parsing]

Parse a tiny JSON subset such as `{"a": 1, "b": [1, 2, {"c": "x"}]}` into Python objects with your own recursive parser (ints, strings, lists, dicts).

Q119 ●●●●●○○○○ 8/10 [precedence, parentheses]

Extend your expression evaluator with parentheses and unary minus. Test 20 expressions against Python's `eval` (only in your tests).

Q120 ●●●●●○○○○ 9/10 [matching, recursion, cost analysis]

Implement a matcher for patterns with `.` and `*` (regex style) recursively. Then reason about how many calls happen for `'a*a*a*a*b'` against `'aaaaaaaaaaa'`, and reduce that with memoization.

CHAPTER 5

Lists: Manipulation, Patterns & Techniques

List me 80% interview problems hote hain. Har question me pehle brute force likho, phir sochho "kya ek pass me ho sakta hai?"

Q121 ●●○○○○○○○○ 2/10 [append, extend, insert, +=]

Predict the result: `a = [1,2]; a.append([3,4]); a.extend([5,6]); a.insert(1, 'x'); a += 'ab'`. What is the difference between `+=` with a string and `append` with a string?

Q122 ●●○○○○○○○○ 2/10 [slicing, rotation]

Rotate a list left by `k` positions using slices only, where `k` may exceed the length. Then rotate right.

Q123 ●●○○○○○○○○ 2/10 [single pass, no sorting]

Find the second largest DISTINCT number in one pass without sorting. What if there is none?

Q124 ●●○○○○○○○○ 2/10 [in-place, iteration]

Remove all occurrences of a value from a list in place. Why does `for x in lst: if x == v: lst.remove(x)` fail? Give two correct solutions.

Q125 ●●○○○○○○○○ 2/10 [swapping, two indices]

Reverse a list in place without `reverse()` or slicing, with a two-index swap loop.

Q126 ●●●○○○○○○○○ 3/10 [dedupe, timing]

Remove duplicates keeping order, three ways. Which is fastest for 100,000 items and why?

Q127 ●●●○○○○○○○○ 3/10 [zip, list of dicts, sorting]

Given lists of names, ages and cities, build a list of dicts. Sort by age descending, then name ascending.

Q128 ●●●○○○○○○○○ 3/10 [nested lists, recursion, stack]

Flatten arbitrarily nested lists like `[1, [2, [3, [4]]]]`: recursively, then iteratively with an explicit stack.

Q129 ●●●○○○○○○○○ 3/10 [chunking]

Split a list into chunks of size `n` (the last may be shorter). Then the reverse problem: split into `n` almost-equal parts whose sizes differ by at most 1.

Q130 ●●●○○○○○○○○ 3/10 [two pointers, merging]

Merge two sorted lists into one sorted list in $O(n + m)$ without `sort`.

Q131 ●●●●○○○○○○ 4/10 [in-place partition]

Move all zeros to the end keeping the order of the non-zeros, in place, in one pass.

Q132 ●●●●○○○○○○ 4/10 [pairs, sets]

Find all unique pairs that sum to a target. Brute force $O(n^2)$ first, then $O(n)$ with a set. Decide how to treat (3, 3) and duplicates.

Q133 ●●●●○○○○○○ 4/10 [rotation, pivot]

Decide whether a list can become sorted by rotating it. [3,4,5,1,2] yes, [2,1,3] no.

Q134 ●●●●○○○○○○ 4/10 [multiset intersection]

Intersection of two lists keeping duplicates as many times as they appear in both ([1,2,2,3] and [2,2,2] give [2,2]).

Q135 ●●●●○○○○○○ 4/10 [majority element, voting]

Find the element appearing more than $n/2$ times, or None. First with a dict, then think about the Boyer-Moore voting idea.

Q136 ●●●●○○○○○○ 4/10 [Kadane, subarrays]

Find the maximum sum of a contiguous subarray. Print the subarray itself too. What should happen when all numbers are negative?

Q137 ●●●●●○○○○○ 5/10 [prefix and suffix products]

For each i , the product of all elements except `nums[i]`, without division and in $O(n)$.

Q138 ●●●●●○○○○○ 5/10 [three-way partition]

Sort a list containing only 0s, 1s and 2s in place in a single pass (Dutch national flag).

Q139 ●●●●●○○○○○ 5/10 [monotonic stack]

For each element, find the next greater element to its right (or -1). Which data structure makes it $O(n)$?

Q140 ●●●●●○○○○○ 5/10 [duplicates, constraints]

A list has $n + 1$ numbers from $1..n$ with one duplicate. Find it without modifying the list and without extra space beyond $O(1)$. Two different approaches.

Q141 ●●●●●○○○○○ 5/10 [intervals, sorting]

Merge overlapping intervals [[1,3], [2,6], [8,10], [15,18]]. Decide what to do with touching intervals [1,2], [2,3] and document your choice.

Q142 ●●●●●○○○○○ 5/10 [next permutation]

Given a list of numbers, produce the next lexicographic permutation in place. What should you return for the last permutation?

Q143 ●●●●●○○○○○ 6/10 [prefix sums, dict]

Find the longest subarray with an equal number of 0s and 1s.

Q144 ●●●●●○○○○ 6/10 [\[quickselect\]](#)

Find the k-th largest element without sorting the whole list (the quickselect idea). What is its worst case?

Q145 ●●●●●○○○○ 6/10 [\[sorting, two pointers\]](#)

Find all unique triplets in a list that sum to zero. How do you avoid duplicate triplets?

Q146 ●●●●●○○○○ 6/10 [\[two pointers, greedy proof\]](#)

Given a list of heights, find two lines that hold the most water. Explain why moving the shorter pointer is safe.

Q147 ●●●●●○○○○ 7/10 [\[prefix maxima, two pointers\]](#)

Trapping rain water from a list of heights. Solve with prefix and suffix maxima, then with two pointers in $O(1)$ extra space.

Q148 ●●●●●○○○○ 7/10 [\[subsequences, DP\]](#)

Length of the longest strictly increasing subsequence: $O(n^2)$ first, then think how a sorted "tails" list gives $O(n \log n)$.

Q149 ●●●●●○○○○ 8/10 [\[deque, monotonic\]](#)

Maximum of every window of size k in $O(n)$ total.

Q150 ●●●●●○○○○ 9/10 [\[binary search on partitions\]](#)

Median of two sorted lists in $O(\log(\min(n, m)))$ time. Explain what a valid partition means before writing any code.

Tuples, Unpacking & Data Modelling

Tuple sirf "immutable list" nahi hai. Yeh Python me records, keys aur multiple returns ka backbone hai.

Q151 ●●○○○○○○○○ 2/10 [tuple syntax]

Predict: `(1), (1,), (), 1, 2, tuple('abc'), (1,2)*2, type((1))`. Which of these surprised you?

Q152 ●●○○○○○○○○ 2/10 [star unpacking]

What are the values in `a, *b, c = range(6)`? Now unpack `first, *_`, `last` from a string and from a dict. What do you get from the dict?

Q153 ●●○○○○○○○○ 2/10 [multiple returns]

Write `min_max_avg(nums)` returning three values. Call it with unpacking and with indexing. What does the function actually return?

Q154 ●●○○○○○○○○ 2/10 [swapping, rotation]

Rotate three variables (a to b to c to a) in one statement. Then reverse a list of 5 using only swap statements.

Q155 ●●●○○○○○○○○ 3/10 [zip and unzip]

Split `[('a',1), ('b',2), ('c',3)]` into two lists using `zip(*...)`. Why does `zip(*zip(*x))` return to the original?

Q156 ●●●○○○○○○○○ 3/10 [tuples as dict keys]

Count how many times each `(x, y)` point is visited on the path `'UURDDL'` starting from the origin.

Q157 ●●●○○○○○○○○ 3/10 [records, fragility]

Represent a student as `(name, marks, city)`. Sort by marks descending, then city ascending. Why is this fragile, and how would you fix it with a class or dict?

Q158 ●●●○○○○○○○○ 3/10 [nested unpacking]

Given `[('Ali', (80, 90)), ('Sara', (70, 85))]`, print each name and average marks using nested unpacking in the `for` header.

Q159 ●●●○○○○○○○○ 3/10 [zip with unequal lengths]

Loop over two lists of unequal length with `zip`, then pad to the longest without `itertools`.

Q160 ●●●○○○○○○○○ 3/10 [lexicographic comparison]

Predict: `(1,2) < (1,3)`, `(1,2) < (1,2,0)`, `(2,) > (1,99,99)`, `('a',1) < ('a','b')`. Explain the comparison rule.

Q161 ●●●●○○○○○ 4/10 [sort keys with mixed directions]

Sort strings by length ascending, then alphabetically DESCENDING. How do you reverse only one part of a key?

Q162 ●●●●○○○○○ 4/10 [min and max with key]

Among a list of points, find the one nearest to the origin; ties go to the smaller x. One expression.

Q163 ●●●●○○○○○ 4/10 [sorting algorithms, swaps]

Implement bubble, selection and insertion sort using tuple swaps. Count comparisons and swaps for each on sorted, reversed and random input.

Q164 ●●●●○○○○○ 4/10 [error signalling]

Write `divide(a, b)` returning `(ok, value_or_message)`. Discuss why exceptions are usually better, then rewrite with exceptions.

Q165 ●●●●○○○○○ 4/10 [variadic functions]

Write `first_last(*args)` returning the first, last, and the middle as a list. Handle 0, 1 and 2 arguments sensibly.

Q166 ●●●●○○○○○ 4/10 [immutability]

Simulate an "immutable point" with tuples: functions `move`, `scale`, `rotate90` returning new tuples. Chain them for a shape of 4 points.

Q167 ●●●●○○○○○ 5/10 [nested dicts from tuples]

Given a list of `(student, subject, mark)` tuples, build `{student: {subject: mark}}`.

Q168 ●●●●○○○○○ 5/10 [transposing data]

Given rows `(name, s1, s2, s3)`, compute each column's average and each row's maximum using `zip`.

Q169 ●●●●○○○○○ 5/10 [adjacent pairs]

For a list, produce all adjacent pairs, all pairs with gap 2, and the differences between them, without indexes.

Q170 ●●●●○○○○○ 5/10 [run grouping]

Group consecutive equal items: `[1,1,2,2,2,3,1,1]` gives `[(1,2), (2,3), (3,1), (1,2)]`. No `itertools`.

Q171 ●●●●○○○○○ 5/10 [polynomials]

Represent a polynomial as a list of `(coefficient, power)` tuples. Write add, multiply and evaluate. How do you keep it normalized?

Q172 ●●●●○○○○○ 5/10 [simulation with tuples]

Model a traffic light as a cycle of `(colour, seconds)` tuples. Simulate 30 seconds and print what is on at each second.

Q173 ●●●●●○○○○ 6/10 [sparse vectors]

Store sparse vectors as dicts `{index: value}`. Write dot product, addition and cosine similarity. Why is this better than lists for a million dimensions?

Q174 ●●●●●○○○○ 6/10 [BFS with tuple states]

A robot on a grid with walls: use tuples as states and a set as visited to find the shortest path length to a target cell.

Q175 ●●●●●○○○○ 6/10 [event processing]

Given a stream of `('add' | 'remove', item, qty)` tuples, produce the final stock and a list of invalid operations (removing more than exists).

Q176 ●●●●●○○○ 7/10 [combinations by hand]

Generate all k-combinations of a list in sorted order without itertools, then all permutations.

Q177 ●●●●●○○○ 7/10 [ranking with ties]

Assign ranks where ties share a rank and the next rank skips (`[100,90,90,80]` gives `[1,2,2,4]`). Return the ranks in the original order.

Q178 ●●●●●○○○ 7/10 [sweep line]

Minimum number of meeting rooms for `(start, end)` meetings. Two ways: sweep line on events, and a sorted list of end times.

Q179 ●●●●●○○○ 8/10 [scheduling structures]

Round-robin tournament for n teams: every pair plays once, and each team plays at most once per round. For `n = 6` print all rounds (the circle method).

Q180 ●●●●●○○○ 9/10 [number theory search]

Find all triples `(a, b, c)` with `a < b < c`, `c <= 500` and `a**2 + b**2 == c**2` in under a second. Then keep only the primitive ones (gcd 1).

Dictionaries: Counting, Grouping, Lookup & Caching

Dict Python ka sabse powerful tool hai. Jab bhi "dhoondhna", "gina", ya "group karna" ho, pehle dict socho.

Q181 ●●○○○○○○○○ 2/10 [dict counting, get, setdefault]

Count the frequency of each character in a string with a plain dict in three ways: `if in`, `.get`, and `.setdefault`.

Q182 ●●○○○○○○○○ 2/10 [sorting items]

Print a dict sorted by value descending, then by key ascending.

Q183 ●●○○○○○○○○ 2/10 [merging]

Merge two dicts where common keys add their values. Then merge two dicts of lists, concatenating on common keys.

Q184 ●●○○○○○○○○ 2/10 [inverting]

Invert `{k: v}` into `{v: [keys]}` handling duplicate values.

Q185 ●●○○○○○○○○ 2/10 [deleting while iterating]

Remove all keys whose value is falsy without raising "dictionary changed size during iteration". Give two ways.

Q186 ●●●○○○○○○○○ 3/10 [dict comprehension]

Build `{n: n**2}` for even n up to 20; then invert it; then filter by a threshold.

Q187 ●●●○○○○○○○○ 3/10 [nested dicts]

Given `(city, year, rain)` records, build `{city: {year: rain}}` and print each city's average rain.

Q188 ●●●○○○○○○○○ 3/10 [two sum]

Two sum with a dict: return the indices of two numbers adding to the target. Why does storing after checking handle duplicates correctly?

Q189 ●●●○○○○○○○○ 3/10 [grouping]

Group words by first letter, then by length; print the groups in sorted order.

Q190 ●●●○○○○○○○○ 3/10 [index building]

Build a word index: for each word, the list of line numbers where it appears in a multi-line text.

Q191 ●●●○○○○○○○○ 3/10 [insertion order]

Show that dicts keep insertion order and use it: remove the oldest item, move a key to the end, get the last inserted key.

Q192 ●●●●○○○○○ 4/10 [ties, counting]

Find the character(s) with the highest frequency (all ties). Then the k most common words with ties broken alphabetically.

Q193 ●●●●○○○○○ 4/10 [prefix sums, dict]

Count the subarrays whose sum is exactly k using a dict of prefix sums.

Q194 ●●●●○○○○○ 4/10 [memoization]

Memoize Fibonacci, then a function of two arguments, using a dict. How would you memoize when an argument is a list?

Q195 ●●●●○○○○○ 4/10 [graphs as dicts]

Build an adjacency list for an undirected graph from an edge list. Find each node's degree and all isolated nodes.

Q196 ●●●●○○○○○ 4/10 [dispatch tables]

Replace a long if/elif calculator with a dict from operator to function. Handle unknown operators and division by zero.

Q197 ●●●●○○○○○ 4/10 [dict views]

Show that `d.keys()` is a live view: change the dict after getting the view and print. Use key views to find common keys, keys only in one dict, and keys whose values differ.

Q198 ●●●●○○○○○ 5/10 [LRU cache]

Implement an LRU cache class with a capacity using a plain dict's insertion order and no imports.

Q199 ●●●●○○○○○ 5/10 [top-k with ties]

From a stream of (`user`, `amount`) purchases, list the top 3 spenders. Make ties deterministic and explain your rule.

Q200 ●●●●○○○○○ 5/10 [recursion, merging]

Write `deep_merge(defaults, overrides)` that merges nested dicts recursively without mutating either input.

Q201 ●●●●○○○○○ 5/10 [flatten and unflatten]

Flatten `{'a': {'b': 1, 'c': {'d': 2}}}` into `{'a.b': 1, 'a.c.d': 2}`, and write the reverse.

Q202 ●●●●○○○○○ 5/10 [sparse grids]

Store a huge sparse grid as `{(r, c): value}`. Write functions for neighbours and for the bounding box of stored cells.

Q203 ●●●●○○○○○ 5/10 [Game of Life, sets and dicts]

Conway's Game of Life on an unbounded board using a set of live cells. Run a glider for 8 steps and print it each time.

Q204 ●●●●●○○○○ 6/10 [graph algorithms]

Detect a cycle in a directed graph given as a dict of lists, using DFS with three states (unvisited, visiting, done).

Q205 ●●●●●○○○○ 6/10 [union-find]

Synonyms [(**'big'**, **'large'**), (**'large'**, **'huge'**), (**'small'**, **'tiny'**)]: are **'big'** and **'huge'** synonyms? Implement a dict-based union-find with path compression.

Q206 ●●●●●○○○○ 6/10 [topological order]

Course schedule: given a prerequisites dict, produce an order to take all courses, or report that it is impossible.

Q207 ●●●●●○○○○ 6/10 [state design]

Leaderboard: events (**player**, **points**) arrive over time; after each event print the top 3 (ties: the one who reached the score first wins). Decide what to store.

Q208 ●●●●●○○○ 7/10 [design]

Design a time-based key-value store: **set(key, value, t)** and **get(key, t)** returning the value at the latest time $\leq t$. Use a dict of lists and binary search written by you.

Q209 ●●●●●●○○ 8/10 [graphs, ordering]

Alien dictionary: given words sorted in an unknown alphabet, derive the letter order. Detect inconsistent input.

Q210 ●●●●●●●○ 9/10 [BFS, bucketing, scale]

Word ladder: shortest transformation from **'hit'** to **'cog'** changing one letter at a time through words in a list. Print the actual path and make it fast for 10,000 words (think of bucket keys like **'h*t'**).

CHAPTER 8

Sets: Uniqueness, Membership & Set Thinking

Set ka asli power: "kya ye pehle dekha hai?" ka jawab $O(1)$ me. Half of graph/grid problems isi pe tiki hain.

Q211 ●●○○○○○○○○ 2/10 [set operations]

Two lists of students take Math and Physics. Print who takes both, either, only Math, and exactly one subject.

Q212 ●●○○○○○○○○ 2/10 [set construction]

Why is `{}` not an empty set? Create an empty set, a set from a string, and a set of tuples. Show which items are not allowed in a set.

Q213 ●●○○○○○○○○ 2/10 [membership speed]

Time `x in list` against `x in set` for 100,000 items and explain the difference.

Q214 ●●○○○○○○○○ 2/10 [add, remove, discard]

What is the difference between `remove` and `discard`? Write a function that toggles an item in a set.

Q215 ●●●○○○○○○○○ 3/10 [subsets, alphabet]

Check whether a sentence is a pangram. Then print the missing letters if it is not.

Q216 ●●●○○○○○○○○ 3/10 [duplicates]

Check whether a list has duplicates in one line. Then find WHICH values are duplicated.

Q217 ●●●○○○○○○○○ 3/10 [symmetric difference]

Given yesterday's and today's file listings as sets, report added, removed and unchanged files.

Q218 ●●●○○○○○○○○ 3/10 [reduce over sets]

Find the common elements of N lists (N arbitrary) in one expression.

Q219 ●●●○○○○○○○○ 3/10 [set comprehension]

List all distinct remainders of squares modulo 7. What do you notice? Try modulo 8 and modulo 13.

Q220 ●●●○○○○○○○○ 3/10 [cycle detection with a set]

A number is happy if repeatedly summing the squares of its digits reaches 1. Detect happy numbers using a set to catch cycles.

Q221 ●●●●○○○○○○ 4/10 [ranges, missing values]

Given a list containing values from $1..n$ with some missing (n up to 10^6), find all missing numbers with a set. Then think how to do it in $O(1)$ extra space by marking.

Q222 ●●●●○○○○○ 4/10 [multiset intersection]

Find the characters common to all words in a list, respecting multiplicity:
['bella', 'label', 'roller'] gives ['e', 'l', 'l'].

Q223 ●●●●○○○○○ 4/10 [similarity]

The similarity of two documents is $|A \cap B| / |A \cup B|$ over their word sets. Rank a list of documents by similarity to a query.

Q224 ●●●●○○○○○ 4/10 [visited sets]

Walk moves like 'RRULLD...' on a grid and return the first position visited twice, or None.

Q225 ●●●●○○○○○ 4/10 [frozenset]

Use frozensets as dict keys to group friend pairs so that (a, b) and (b, a) count as the same pair.

Q226 ●●●●○○○○○ 5/10 [order-preserving set logic]

Given two lists, return items in the first not in the second, and vice versa, with no duplicates and in original order.

Q227 ●●●●○○○○○ 5/10 [equality of graphs]

Check whether two lists of pairs describe the same undirected graph regardless of order and direction.

Q228 ●●●●○○○○○ 5/10 [Sudoku validation]

Validate a partially filled 9x9 Sudoku with sets: rows, columns and 3x3 boxes.

Q229 ●●●●○○○○○ 5/10 [consecutive run]

Longest run of consecutive integers in an unsorted list in $O(n)$, no sorting. Start counting only from run beginnings, and explain why that gives $O(n)$.

Q230 ●●●●○○○○○ 5/10 [subsets]

Generate all subsets of a list recursively, iteratively and with bitmasks. Compare the orderings.

Q231 ●●●●○○○○○ 5/10 [substrings, memory]

Count the distinct substrings of a string of length up to 200 using a set. Discuss the memory use.

Q232 ●●●●○○○○○ 6/10 [pairs with difference]

Count unique pairs of values with difference exactly k using a set only.

Q233 ●●●●○○○○○ 6/10 [reachable sums]

Can any subset of a list sum to target? Keep a set of reachable sums and update it per item, with no recursion.

Q234 ●●●●○○○○○ 6/10 [sliding window with a set]

Is there a pair of equal values whose indices differ by at most k? Use a sliding window set.

Q235 ●●●●●○○○ 6/10 [connected components]

Count connected components of an undirected graph given as an edge list, using sets for visited.

Q236 ●●●●●○○○ 6/10 [flood fill]

Count islands in a grid of '1' and '0' using an iterative flood fill and a visited set.

Q237 ●●●●●○○○ 7/10 [set cover]

Given a list of sets, find the minimum number of them whose union covers all elements (brute force for small inputs). Construct an example where the greedy "largest first" strategy fails.

Q238 ●●●●●○○○ 7/10 [linked structures]

Detect whether a structure encoded as `{node: next}` has a cycle and return the node where the cycle starts, using a set. Then think how to do it with $O(1)$ memory.

Q239 ●●●●●○○○ 8/10 [window with counts]

Longest subarray with at most k distinct values, using a window with dict counts.

Q240 ●●●●●○○○ 9/10 [constraint puzzle]

Three friends, three pets, three colours: invent five clues so that exactly one assignment satisfies all of them. Solve it with sets and permutations you generate by hand.

CHAPTER 9

Loops, Conditions & Simulation

Loop likhna aasan hai, sahi loop likhna kala hai. Boundaries, invariants aur termination pe dhyan do.

Q241 ●●○○○○○○○○ 2/10 [for, range, patterns]

For $n = 5$ print: a right triangle, an inverted triangle, a hollow square, and the number pyramid (1 / 121 / 12321).

Q242 ●●○○○○○○○○ 2/10 [while, state]

Given a secret number and a list of guesses, print 'higher', 'lower' or 'correct after k tries'.

Q243 ●●○○○○○○○○ 2/10 [for-else]

Use `for ... else` to test whether a number is prime. Explain what `else` on a loop means in your own words.

Q244 ●●○○○○○○○○ 2/10 [nested loops]

Print all pairs (i, j) with $1 \leq i < j \leq 5$ and count them. Then all triples $i < j < k$. What are the formulas for the counts?

Q245 ●●○○○○○○○○ 2/10 [enumerate]

Print a list with 1-based numbering, marking every 3rd item with `*`.

Q246 ●●●○○○○○○○ 3/10 [accumulators]

Compute compound interest year by year until the money doubles and report the year. Print money with 2 decimals.

Q247 ●●●○○○○○○○ 3/10 [modifying while iterating]

Demonstrate the danger of removing evens from `[1,2,2,3,4,4,6]` while iterating, and give three safe alternatives.

Q248 ●●●○○○○○○○ 3/10 [continue]

Print numbers 1 to 50, skipping multiples of 7 and any number containing the digit 7.

Q249 ●●●○○○○○○○ 3/10 [sentinels, iterators]

Read numbers until the entry 'done' (simulate input with an iterator and `next()`). Compute the average and handle zero entries.

Q250 ●●●○○○○○○○ 3/10 [alignment]

Print a neatly aligned 12x12 multiplication table with a header row and column.

Q251 ●●●○○○○○○○ 3/10 [triangles]

Print Pascal's triangle of n rows, building each row only from the previous one.

Q252 ●●●●○○○○○ 4/10 [invariants, binary search]

Write binary search with a `while` loop and state the invariant (what is always true at the top of each iteration). Test empty list, one item, duplicates and absent items.

Q253 ●●●●○○○○○ 4/10 [generalization]

Generalize FizzBuzz: given `{3: 'Fizz', 5: 'Buzz', 7: 'Bazz'}` produce the outputs for 1 to 105 without one `if` per rule.

Q254 ●●●●○○○○○ 4/10 [state machine]

Vending machine: given coin insertions and a selection, decide change or refund. Use a dict of prices and cover exact change and insufficient funds.

Q255 ●●●●○○○○○ 4/10 [2D loops, spiral]

Print the numbers 1..n squared in a spiral matrix ($n = 4$). Then reverse the direction.

Q256 ●●●●○○○○○ 4/10 [queue simulation]

Customers arrive with `(arrival_time, service_time)` and one teller serves them in order. Compute each customer's waiting time and the average.

Q257 ●●●●○○○○○ 4/10 [inversions]

Count the swaps bubble sort needs. Then count the inversions by brute force and confirm they match. Why do they match?

Q258 ●●●●○○○○○ 5/10 [algorithm from scratch]

Find the weekday of a date `(d, m, y)` with your own algorithm (no `datetime`). Verify on 10 known dates.

Q259 ●●●●○○○○○ 5/10 [formatting a calendar]

Print a month calendar like `cal`, given the weekday of the 1st and the number of days.

Q260 ●●●●○○○○○ 5/10 [game logic]

Tic-tac-toe winner detection for a 3x3 board given as strings, then for NxN. Detect draws and invalid boards (for example both players winning).

Q261 ●●●●○○○○○ 5/10 [Josephus]

n people stand in a circle and every k -th is eliminated. Return the survivor: simulate with a list, then find the recurrence.

Q262 ●●●●○○○○○ 5/10 [search loops]

Find all integer solutions to $x^2 + y^2 = z^2$ with $z \leq 100$. Remove duplicate `(y, x)` pairs and cut loop work with a set of squares.

Q263 ●●●●○○○○○ 5/10 [digit patterns]

Print all 4-digit numbers whose digits are strictly increasing, and count them. Predict the count with a formula first.

Q264 ●●●●●○○○○ 6/10 [\[generation with constraints\]](#)

Generate all binary strings of length n with no two consecutive 1s. How does the count grow with n ?

Q265 ●●●●●○○○○ 6/10 [\[Langton's ant\]](#)

Simulate Langton's ant for 200 steps on a grid stored as a set of black cells and print the bounding box of the black cells.

Q266 ●●●●●○○○○ 6/10 [\[elevator\]](#)

Simulate an elevator serving floor requests in SCAN order (go up serving everything, then down). Output the visiting order and the total distance.

Q267 ●●●●●○○○○ 6/10 [\[pivot index\]](#)

Find the index where the sum on the left equals the sum on the right. Then find all such indices.

Q268 ●●●●●○○○○ 7/10 [\[search, precision\]](#)

Game of 24: given four numbers, decide whether $+$ $-$ $*$ $/$ and parentheses can make 24. Recurse over pairs and handle division carefully.

Q269 ●●●●●●○○○ 8/10 [\[minesweeper\]](#)

Minesweeper: compute the numbers for a board, then implement the "click" that flood-reveals zero cells. Verify against a hand-made board.

Q270 ●●●●●●●○○ 9/10 [\[state-space search\]](#)

8-puzzle: check whether a start state is solvable using inversion parity, and find the shortest solution by BFS. Print the sequence of boards.

Functions: Design, Scope, Closures & Decorators

Function sirf code reuse nahi, thinking ka unit hai. Yahan tum functions ko objects ki tarah treat karna seekhoge.

Q271 ●●○○○○○○○○ 2/10 [defaults, keyword arguments]

Write `power(base, exp=2)`. Call it with positional, keyword and mixed arguments. Which calls are errors and why?

Q272 ●●○○○○○○○○ 2/10 [return None]

What does a function without `return` give back? Use the result of a print-only function inside an expression and predict what happens.

Q273 ●●○○○○○○○○ 2/10 [*args]

Write `my_max(*args, key=None)` mimicking `max` for one or more arguments, without using `max`.

Q274 ●●○○○○○○○○ 2/10 [**kwargs]

Write `make_tag(name, **attrs)` producing `<a href="x" class="y">`. Convert underscores to hyphens (`data_id` becomes `data-id`) and think about attribute order.

Q275 ●●●○○○○○○○○ 3/10 [LEGB]

Explain the LEGB scope rule with four examples of your own, one per letter. Include one where an inner function reads and then assigns a variable of the outer function.

Q276 ●●●○○○○○○○○ 3/10 [first-class functions]

Store functions in a list and in a dict and call them by name from a user string. Write `apply_n(f, x, n)`.

Q277 ●●●○○○○○○○○ 3/10 [lambda, sorting]

Sort a list of dicts by 'age' then 'name' with lambdas. Then rewrite without lambda using your own small helper.

Q278 ●●●○○○○○○○○ 3/10 [introspection]

Write functions with docstrings and annotations, print `f.__doc__` and `f.__annotations__`, then write `describe(f)` that prints name, parameter names and defaults using `f.__code__` and `f.__defaults__`.

Q279 ●●●○○○○○○○○ 3/10 [recursion basics]

Write factorial, digit sum, power and string reversal recursively, then convert each to a loop.

Q280 ●●●○○○○○○○○ 3/10 [pure functions]

Which of these are pure and why: `sorted`, `list.sort`, `print`, `len`, `random.random`, a function that appends to a global? Rewrite one impure function as a pure one.

Q281 ●●●●○○○○○ 4/10 [keyword-only arguments]

Design `connect(host, port, *, timeout=5, retries=3)`. What error does `connect('h', 80, 10)` give? Why is this API safer?

Q282 ●●●●○○○○○ 4/10 [argument unpacking]

Call a function with values from a list and a dict using `f(*args, **kwargs)`. What happens if a key is both a positional value and in kwargs?

Q283 ●●●●○○○○○ 4/10 [closures]

Write `make_counter()` and `make_multiplier(n)`. Then an `accumulator` that remembers the total of everything passed to it.

Q284 ●●●●○○○○○ 4/10 [higher-order functions]

Write your own `map`, `filter` and `reduce` (with an optional initial value) using loops only.

Q285 ●●●●○○○○○ 4/10 [decorators]

Write `@timer` (prints elapsed time) and `@logged` (prints the function name and arguments). Stack them. In which order do they run?

Q286 ●●●●○○○○○ 4/10 [function factories, composition]

Write `compose(f, g, h)` and `pipe(f, g, h)`. Use one to build `clean = strip, lower, remove punctuation`.

Q287 ●●●●○○○○○ 5/10 [decorators with arguments]

Write `@retry(times=3)` that re-runs a function raising an exception, and `@memoize` that supports keyword arguments. Preserve the function's name and docstring.

Q288 ●●●●○○○○○ 5/10 [partial application]

Implement `partial(f, *a, **k)` yourself. Then derive `square` and `cube` from `pow`.

Q289 ●●●●○○○○○ 5/10 [recursion limits]

Why does a recursive `sum_to(n)` fail near `n = 1000`? Show the error, raise the limit, then rewrite iteratively. Which recursions are worth converting?

Q290 ●●●●○○○○○ 5/10 [generators as functions]

Write `fib()` as an infinite generator and `take(n, gen)`. Then a `primes()` generator.

Q291 ●●●●○○○○○ 5/10 [decorators, validation]

Write `@typecheck(int, int)` that raises `TypeError` when arguments do not match the given types.

Q292 ●●●●○○○○○ 5/10 [callbacks]

Write a mini event system: `on(event, fn)` and `emit(event, *args)`, where several functions can subscribe to one event.

Q293 ●●●●●○○○○ 6/10 [currying]

Convert `add(a, b, c)` into `add(1)(2)(3)` manually, then write `curry(f)` that works for any number of positional parameters (use `f.__code__.co_argcount`).

Q294 ●●●●●○○○○ 6/10 [memoization]

Count paths in an $n \times m$ grid moving right or down: naive recursion, dict memo, bottom-up. Time each for 15×15 .

Q295 ●●●●●○○○○ 6/10 [mutual recursion]

Write `is_even` and `is_odd` via mutual recursion. Then sketch a recursive-descent parser for expressions with two mutually recursive functions.

Q296 ●●●●●○○○○ 6/10 [recursion over structures]

Sum all numbers in an arbitrarily nested list/dict structure, ignoring strings and booleans.

Q297 ●●●●●○○○ 7/10 [anonymous recursion]

Write a recursive function that does NOT refer to its own name (pass itself as a parameter). Explain how anonymous recursion works.

Q298 ●●●●●○○○ 7/10 [trampolining]

Convert a deeply recursive function into a trampoline that never overflows the stack: functions return either a value or a thunk for the next step.

Q299 ●●●●●○○○ 8/10 [testable design]

Design `retry_with_backoff(fn, attempts, base_delay)` that returns `fn`'s result or raises the last error. Make it testable by injecting `sleep` as a parameter, and write three tests.

Q300 ●●●●●○○○ 9/10 [self-reference puzzles]

Write a function `f` such that `f(f)` returns something different from `f(1)` and never crashes. Then a function such that `f()()()` works forever and reports how many times it was called.

CHAPTER 11

Comprehensions, Generators & the Iteration Protocol

Iteration Python ka DNA hai. Jab tum "lazy" sochne lagoge, tumhara code chhota aur fast dono ho jayega.

Q301 ●●○○○○○○○○ 2/10 [list comprehensions]

Rewrite five loops as comprehensions: squares of evens, uppercase of long words, pairs `(i, j)` with `i != j`, a flattening, and a dict from two lists.

Q302 ●●○○○○○○○○ 2/10 [conditional expression vs filter]

Compare `[x if x > 0 else -x for x in a]` with `[x for x in a if x > 0]`. Predict their lengths and write the rule for where `if` goes.

Q303 ●●○○○○○○○○ 2/10 [set and dict comprehensions]

Given words, build a set of lengths, a dict word to length, and a dict length to list of words (this needs more than one comprehension).

Q304 ●●○○○○○○○○ 2/10 [generator expressions]

Compare `sum([x*x for x in range(10**6)])` with the generator version for memory and time. Where can a generator expression NOT replace a list?

Q305 ●●●○○○○○○○ 3/10 [nested comprehensions]

Build a multiplication table as a list of lists in one comprehension. Transpose it in one comprehension without `zip`.

Q306 ●●●○○○○○○○ 3/10 [readability]

Rewrite this with loops and decide which is better: `[(x, y) for x in range(20) for y in range(x) if (x + y) % 3 == 0 if x * y > 10]`.

Q307 ●●●○○○○○○○ 3/10 [generator functions]

Write a generator `countdown(n)` and use it with `for`, `list()`, `sum`, `in` and `next()`. What happens after exhaustion?

Q308 ●●●○○○○○○○ 3/10 [generator state]

Create a generator, consume two items, then loop over it with `for`. What gets printed and why?

Q309 ●●●○○○○○○○ 3/10 [zip and generators]

Use two generators of different lengths in `zip()`. Which generator ends up with an extra consumed item?

Q310 ●●●○○○○○○○ 3/10 [manual iteration]

Iterate a list manually with `iter()` and `next()`, catching `StopIteration`. Then use `next(it, None)`.

Q311 ●●●●○○○○○ 4/10 [batching]

Write `chunks(iterable, n)` yielding lists of `n` items. It must work on any iterable, including generators.

Q312 ●●●●○○○○○ 4/10 [sliding windows]

Write `windows(seq, k)` yielding overlapping windows of size `k`. Use it for a moving average.

Q313 ●●●●○○○○○ 4/10 [yield from]

Flatten nested lists with a recursive generator using `yield from`. Why is this better than returning a list?

Q314 ●●●●○○○○○ 4/10 [infinite streams]

Write `naturals()`, `evens(gen)` and `take_while(pred, gen)` and combine them to print the even numbers below 30.

Q315 ●●●●○○○○○ 4/10 [iterator classes]

Write `class Range(start, stop, step)` with `__iter__` and `__next__`. Make it reusable (iterating twice works) by separating the iterable from its iterator.

Q316 ●●●●○○○○○ 4/10 [lazy pipelines]

Read lines lazily (simulate with a generator), strip, drop comments, split into words and count them, all with chained generators. Show that nothing runs until consumption.

Q317 ●●●●○○○○○ 5/10 [send]

Write a generator `running_avg()` that receives numbers with `send()` and yields the running average. Why must you call `next()` first?

Q318 ●●●●○○○○○ 5/10 [generator cleanup]

What happens if you `break` out of a loop over a generator that has `try/finally`? Demonstrate with `.close()`.

Q319 ●●●●○○○○○ 5/10 [streaming dedupe]

Write `unique(iterable)` yielding items not seen before, then `unique_by(iterable, key)`. How do you handle unhashable items?

Q320 ●●●●○○○○○ 5/10 [lazy merging]

Write `merge(*sorted_iterables)` merging any number of sorted streams lazily. Keep only the "current head" of each stream.

Q321 ●●●●○○○○○ 5/10 [prime streams]

Write an infinite prime generator that tests candidates using the primes found so far. Compare it to a sieve for the first 1000 primes.

Q322 ●●●●○○○○○ 5/10 [lazy permutations]

Generate permutations of a list lazily with recursion and `yield`. Count them without storing.

Q323 ●●●●●○○○○ 6/10 [rebuild the toolbox]

Implement `zip_longest`, `cycle`, `repeat`, `accumulate` and `chain` from scratch as generators.

Q324 ●●●●●○○○○ 6/10 [lazy backtracking]

Generate all valid parentheses combinations for n pairs lazily, in lexicographic order. Print the first 5 for $n = 10$ without computing the rest.

Q325 ●●●●●○○○○ 6/10 [tree traversals]

With a tree as nested tuples `(value, left, right)`, write in-order, pre-order, post-order and level-order traversal generators.

Q326 ●●●●●○○○○ 6/10 [push pipelines]

Build a pipeline `producer`, `filter`, `consumer` with generators using `send()` (push style) instead of pull style.

Q327 ●●●●●○○○ 7/10 [lazy streams]

Implement a `Stream` class with `.map(f)`, `.filter(p)`, `.take(n)` chained by method calls that produces values only when `list()` is called.

Q328 ●●●●●○○○ 7/10 [old-style iteration]

Explain why `for x in obj` works when the class only defines `__getitem__`. Build such a class and show its quirks.

Q329 ●●●●●○○○ 8/10 [pruned search]

Generate all subsets of items with total weight at most W lazily, pruning early. Compare how many subsets you examine against 2 to the power n .

Q330 ●●●●●○○○ 9/10 [merging streams]

Write a generator yielding Hamming numbers (only prime factors 2, 3, 5) in increasing order, with no repeats and no sorting a big list, in $O(1)$ work per number.

CHAPTER 12

Sorting & Searching: Keys, Stability & Binary Search Thinking

Sorting sirf sort() nahi hai. Key kaise banate hain aur binary search kab lagti hai, yahi asli skill hai.

Q331 ●●○○○○○○○○ 2/10 [sorted, key]

Sort names by length then alphabetically, sort case-insensitively, and sort numeric strings numerically ('10', '9', '2').

Q332 ●●○○○○○○○○ 2/10 [stability]

Sort people by city then by age using two separate sorts instead of a tuple key. Why does this work? Which sort goes first?

Q333 ●●○○○○○○○○ 2/10 [partial ordering]

Find the 3 smallest and 3 largest numbers without sorting the whole list.

Q334 ●●○○○○○○○○ 2/10 [linear vs binary search]

Return all indices of a target by linear search. Then return the first and last index using binary search written by you.

Q335 ●●●○○○○○○○ 3/10 [custom order]

Sort the characters of a string by a custom alphabet order given as another string ('bca').

Q336 ●●●○○○○○○○ 3/10 [multi-key sort]

Sort students by marks descending; ties by name ascending; ties by roll number ascending, with one sort call.

Q337 ●●●○○○○○○○ 3/10 [basic sorts]

Write insertion sort and selection sort, count comparisons, and decide which is better on nearly sorted data.

Q338 ●●●○○○○○○○ 3/10 [k-th of two lists]

Given two sorted lists, find the k-th smallest of their union without merging fully.

Q339 ●●●○○○○○○○ 3/10 [insertion points]

Find the insertion position of x in a sorted list, then floor and ceiling, then the number of occurrences with two binary searches.

Q340 ●●●●○○○○○○ 4/10 [merge sort]

Implement merge sort recursively, then bottom-up. Count inversions in the same pass.

Q341 ●●●●○○○○○○ 4/10 [quicksort]

Implement quicksort with (a) first-element pivot, (b) random pivot, (c) median of three. Show an input that makes (a) quadratic.

Q342 ●●●●○○○○○ 4/10 [counting sort]

Sort integers from 0 to 100 with counting sort. Then sort exam records by mark with a stable counting sort.

Q343 ●●●●○○○○○ 4/10 [frequency sorting]

Sort elements by frequency descending, ties by value ascending. Then keep first-appearance order for ties.

Q344 ●●●●○○○○○ 4/10 [custom comparison]

Arrange numbers into the largest concatenation (`[3,30,34,5,9]` gives `'9534330'`). Explain the comparison rule and implement it.

Q345 ●●●●○○○○○ 4/10 [natural sort]

Sort `['a10', 'a2', 'a1']` into natural order `['a1', 'a2', 'a10']`.

Q346 ●●●●○○○○○ 5/10 [rotated arrays]

Search a target in a rotated sorted list (`[4,5,6,7,0,1,2]`) in $O(\log n)$.

Q347 ●●●●○○○○○ 5/10 [peaks]

Find a peak element (greater than its neighbours) in $O(\log n)$. Why does binary search work when the list is not sorted?

Q348 ●●●●○○○○○ 5/10 [binary search on the answer]

Given plank lengths and k , find the maximum length L such that you can cut k pieces of length L . What makes a problem binary-searchable on its answer?

Q349 ●●●●○○○○○ 5/10 [numerical search]

Approximate `sqrt(x)` to 6 decimals using binary search, then Newton's method. Compare the iteration counts.

Q350 ●●●●○○○○○ 5/10 [sorted matrices]

Search a value in a matrix sorted along rows and columns in $O(n + m)$.

Q351 ●●●●○○○○○ 5/10 [closest values]

For a sorted list and a target x , find the k values closest to x .

Q352 ●●●●○○○○○ 5/10 [heaps]

Implement a min-heap class with push, pop and peek using a list and sift up/down. Then implement heap sort.

Q353 ●●●●○○○○○ 6/10 [greedy with sorting]

Maximum number of non-overlapping intervals (activity selection). Construct a counter-example that breaks "sort by start".

Q354 ●●●●●○○○○ 6/10 [sorting formats]

Sort 'HH:MM' strings, times with AM/PM, and dates 'dd-mm-yyyy' correctly without `datetime`.

Q355 ●●●●●○○○○ 6/10 [k-way merge]

You have 10 sorted lists ("files") of 1M numbers each and memory for about 1M numbers total. Design a k-way merge that reads chunk by chunk.

Q356 ●●●●●○○○○ 6/10 [matrix search]

Find the k-th smallest element in an $n \times n$ matrix with sorted rows and columns without flattening it.

Q357 ●●●●●○○○○ 7/10 [two heaps]

Maintain the running median as numbers arrive using two heaps (yours or `heapq`).

Q358 ●●●●●○○○○ 7/10 [merge-sort counting]

For each element count how many elements to its right are smaller, in $O(n \log n)$.

Q359 ●●●●●○○○○ 8/10 [DP with binary search]

Weighted job scheduling: choose non-overlapping jobs to maximize profit using sorting, binary search and DP.

Q360 ●●●●●○○○○ 9/10 [inversions, proof]

Find the minimum number of adjacent swaps to sort a list and reconstruct one sequence of swaps. Prove your count using the inversion count.

CHAPTER 13

Recursion & Backtracking

Recursion ka matlab: badi problem = chhoti wahi problem + ek step. Base case pehle likho, hamesha.

Q361 ●●○○○○○○○○ 2/10 [base case, recursion]

Write recursively: sum of a list, length of a string, maximum of a list, count of a digit in a number. State the base case of each in one line.

Q362 ●●○○○○○○○○ 2/10 [call trees, memoization]

Draw the call tree of `fib(5)` on paper. How many calls happen? How many are repeats? Then count the calls for `fib(30)` with a counter, before and after memoization.

Q363 ●●○○○○○○○○ 2/10 [string recursion]

Reverse a string, check for a palindrome, and remove adjacent duplicates ('abbaca' gives 'ca'), all recursively.

Q364 ●●○○○○○○○○ 2/10 [divide and conquer]

Fast exponentiation `x**n` recursively in $O(\log n)$. Handle negative n and $n = 0$.

Q365 ●●●○○○○○○○ 3/10 [Tower of Hanoi]

Print the moves for $n = 4$. Count the moves for $n = 20$ without printing. Find the formula and prove it by induction in one sentence.

Q366 ●●●○○○○○○○ 3/10 [binary search recursion]

Write recursive binary search and print the searched range at each call.

Q367 ●●●○○○○○○○ 3/10 [include/exclude decisions]

Generate all subsets of a list recursively. Then only those with sum equal to a target.

Q368 ●●●○○○○○○○ 3/10 [permutations]

All permutations of a string with distinct letters, then with duplicate letters WITHOUT duplicates in the output.

Q369 ●●●○○○○○○○ 3/10 [digit generation]

Generate all n -digit numbers whose digits sum to s .

Q370 ●●●○○○○○○○ 3/10 [flood fill]

Paint-bucket fill on a 2D grid recursively. What limit will you hit on a 1000×1000 grid? Rewrite it iteratively.

Q371 ●●●●○○○○○○ 4/10 [combinations]

All k -combinations of n items without itertools. Then combinations that allow repeated items.

Q372 ●●●●○○○○○ 4/10 [parentheses]

Generate all balanced parentheses of n pairs. Then check whether a string with `?` wildcards can be completed into a valid one.

Q373 ●●●●○○○○○ 4/10 [counting ways]

Count the ways to make an amount with unlimited coins, then the minimum number of coins. First plain recursion, then memoized.

Q374 ●●●●○○○○○ 4/10 [stairs]

Ways to climb n stairs with steps 1, 2 or 3; then with a list of forbidden steps.

Q375 ●●●●○○○○○ 4/10 [pruned enumeration]

Phone keypad combinations for '23' ('ad', 'ae', ...). Then keep only those that are English words from a given list, pruning early.

Q376 ●●●●○○○○○ 4/10 [segmentation]

Can a string be segmented into dictionary words? Then return all segmentations.

Q377 ●●●●○○○○○ 5/10 [constraint search]

Place n queens. Count the solutions for $n = 1$ to 8 and print one board.

Q378 ●●●●○○○○○ 5/10 [maze DFS]

Find a path from S to E in a grid maze with recursion and return the path. Then find the shortest path with BFS. Compare.

Q379 ●●●●○○○○○ 5/10 [partitioning]

Partition a string into palindromes: return all ways.

Q380 ●●●●○○○○○ 5/10 [subset sum variants]

Split a list into two subsets of equal sum: plain recursion, then memoized, then the reachable-sums method.

Q381 ●●●●○○○○○ 5/10 [state counting]

Count the squares a knight can reach in exactly k moves on an 8x8 board from a given square.

Q382 ●●●●○○○○○ 5/10 [expression generation]

Insert `+`, `-` or nothing between the digits of '123' to reach a target. Return all expressions.

Q383 ●●●●○○○○○ 6/10 [Sudoku]

Solve a 4x4 Sudoku by backtracking, then generalize to 9x9.

Q384 ●●●●○○○○○ 6/10 [Gray code]

Generate the Gray code sequence for n bits recursively. Verify that consecutive numbers differ in exactly one bit.

Q385 ●●●●●○○○ 6/10 [\[path enumeration\]](#)

Find ALL paths from top-left to bottom-right of a grid moving in 4 directions without revisiting a cell. Count them for a 4x4 grid.

Q386 ●●●●●○○○ 7/10 [\[memoized matching\]](#)

Wildcard matching with `*` and `?` via memoized recursion. Count the recursive calls with and without memoization.

Q387 ●●●●●○○○ 7/10 [\[decoding\]](#)

Number of ways to decode `'11106'` with A=1 to Z=26, then list the decodings. Handle `'0'` correctly.

Q388 ●●●●●○○○ 8/10 [\[graph colouring\]](#)

Colour a graph with at most `m` colours so adjacent vertices differ, by backtracking. Report when it is impossible.

Q389 ●●●●●○○○ 8/10 [\[constraint ordering\]](#)

Fill a small crossword grid from a word list with backtracking. Choose the most constrained slot first and measure the gain.

Q390 ●●●●●○○○ 9/10 [\[heuristic search\]](#)

Knight's tour on a 6x6 board via backtracking with Warnsdorff's heuristic. Compare the number of backtracks with and without the heuristic.

CHAPTER 14

Classes & Object-Oriented Design

Class tab likho jab data aur uske rules saath rehte hain. Har question me pehle socho: "kaun kis cheez ka maalik hai?"

Q391 ●●○○○○○○○○ 2/10 [class basics, __str__, __repr__]

Create `class Student` with a name and a list of marks. Add `add_mark`, `average` and `grade`. Print with `__str__` and `__repr__` and show the difference by printing a list of students.

Q392 ●●○○○○○○○○ 2/10 [class vs instance attributes]

A class attribute `count = 0` is incremented in `__init__` either with `self.count += 1` or `Student.count += 1`. Which one works as a counter and why?

Q393 ●●○○○○○○○○ 2/10 [self, bound methods]

What is `self`? Call a method as `Student.average(s)` instead of `s.average()`. Then store the bound method in a variable and call it later.

Q394 ●●○○○○○○○○ 2/10 [encapsulation, exceptions]

Write a `BankAccount` that refuses negative deposits and overdrafts (raise exceptions) and exposes a read-only balance property.

Q395 ●●●○○○○○○○○ 3/10 [dunder methods]

Write `Vector2D` with `+`, `-`, scalar `*`, `==`, `abs`, `str`, and unpacking (`x, y = v`). What does `2 * v` need?

Q396 ●●●○○○○○○○○ 3/10 [properties]

Write a `Temperature` class with settable `celsius` and `fahrenheit` properties and a single source of truth.

Q397 ●●●○○○○○○○○ 3/10 [inheritance, polymorphism]

Shape with `Circle`, `Rectangle` and `Square`. Give each an `area()`, then write a function that returns the biggest shape in a list. Should `Square` inherit from `Rectangle`? Argue both sides.

Q398 ●●●○○○○○○○○ 3/10 [super]

Write `Animal` and `Dog` with `super().__init__`. Show the actual bug when you forget it.

Q399 ●●●○○○○○○○○ 3/10 [classmethod, staticmethod]

Alternative constructors: `Date.from_string('2026-01-05')`. Add `is_leap(year)` as a staticmethod. When do you use a classmethod and when a staticmethod?

Q400 ●●●○○○○○○○○ 3/10 [ordering]

Make `Card` objects comparable and sortable (rank, then suit). Which dunder methods are the minimum needed? Sort a hand.

Q401 ●●●●○○○○○○ 4/10 [containers]

Write `Stack` and `Queue` classes with `push/pop`, `__len__`, `__bool__`, `__iter__` and `__repr__`. Raise a custom exception when popping from an empty container.

Q402 ●●●●○○○○○○ 4/10 [__eq__ and __hash__]

Make two `Point` objects with equal coordinates equal and usable as dict keys and set members. What breaks if you define `__eq__` but not `__hash__`?

Q403 ●●●●○○○○○○ 4/10 [composition, rules]

Model a `Library` with `Book`, `Member` and loans. Rules: at most 3 books per member, no lending a book that is already lent, fines for late returns. Sketch the classes on paper before coding.

Q404 ●●●●○○○○○○ 4/10 [__getitem__, operators]

Write a `Matrix` class supporting `m[i][j]`, `m[i, j]`, `len`, `+`, `*` (matrix product), `transpose()` and pretty printing.

Q405 ●●●●○○○○○○ 4/10 [operator overloading, formatting]

Implement `Polynomial` with `+`, `*`, `__call__` (evaluate), `derivative` and a `__str__` that prints $3x^2 - 2x + 1$ correctly (signs, coefficient 1, zero terms).

Q406 ●●●●○○○○○○ 4/10 [context managers]

Write class-based context managers `Timer` and `Suppress(*exceptions)` using `__enter__` and `__exit__`. What does returning `True` from `__exit__` do?

Q407 ●●●●●○○○○○ 5/10 [abstract classes]

Make a base class whose methods raise `NotImplementedError`. Then use `abc` so it cannot be instantiated. Which approach is better and why?

Q408 ●●●●●○○○○○ 5/10 [multiple inheritance, MRO]

`class D(B, C)` where both `B` and `C` inherit from `A`. Predict the method resolution order and the outputs when all of them call `super()`.

Q409 ●●●●●○○○○○ 5/10 [memory]

Compare the memory of 100,000 instances with and without `__slots__` using `sys.getsizeof`. Careful: it is shallow.

Q410 ●●●●●○○○○○ 5/10 [state machines]

Model an order (NEW, PAID, SHIPPED, DELIVERED, CANCELLED) with allowed transitions. Invalid transitions raise errors. Implement with if-chains and with a dict of transitions.

Q411 ●●●●●○○○○○ 5/10 [linked list class]

`LinkedList` with `__iter__`, `append`, `prepend`, `remove(value)`, in-place `reverse`, `__len__` and `__contains__`.

Q412 ●●●●●○○○○ 5/10 [observer pattern]

A `Stock` class with subscribers (callables) notified on price change. What breaks if a subscriber unsubscribes during notification?

Q413 ●●●●●○○○○ 6/10 [property factories]

Write `positive_int('age')` that returns a property with validation; use it in two different classes.

Q414 ●●●●●○○○○ 6/10 [BST]

Binary search tree with insert, search, delete (three cases), in-order iterator, height and `__len__`.

Q415 ●●●●●○○○○ 6/10 [LRU with linked nodes]

A doubly linked list plus a dict gives an $O(1)$ LRU cache class. Test the eviction order thoroughly.

Q416 ●●●●●○○○○ 6/10 [inheritance in a system]

Extend `BankAccount` into Savings (interest) and Checking (overdraft limit), with a `Bank` class handling transfers atomically: either both sides change or neither.

Q417 ●●●●●○○○ 7/10 [immutability]

Create an immutable `Money` class (currency, amount) with arithmetic, refusing mixed currencies, hashable, sortable, and with no attribute assignment after creation.

Q418 ●●●●●○○○ 7/10 [full numeric type]

Write a `Fraction` class with normalization, arithmetic, comparisons, `__float__` and `__str__`. Make both `Fraction(1, 2) + 1` and `1 + Fraction(1, 2)` work.

Q419 ●●●●●○○○ 8/10 [registry pattern]

Build a tiny command dispatcher where classes register themselves in a dict via a decorator `@register('name')`. Add a new command without touching the dispatcher code.

Q420 ●●●●●●○○ 9/10 [design]

Design a text-adventure engine: rooms, items, a player, and commands parsed from strings ('go north', 'take lamp'), with save/load of state to a dict. Focus on class relationships, not the story.

Exceptions & File Handling

Program tab tak "sahi" nahi jab tak galat input pe bhi gracefully behave na kare. Yahan tum defensive coding seekhoge.

Q421 ●●○○○○○○○○ 2/10 [try/except]

Handle `ValueError` and `ZeroDivisionError` separately in a calculator that reads '`a op b`' strings. Use different messages.

Q422 ●●○○○○○○○○ 2/10 [finally, return]

Predict what a function does with `try: return 1 finally: print('x')`. Then with a `return` inside `finally` as well.

Q423 ●●○○○○○○○○ 2/10 [reading, writing]

Write a list of lines to a file, then read them back with `read()`, `readline()`, `readlines()` and iteration. Which is memory-friendly for huge files?

Q424 ●●○○○○○○○○ 2/10 [with]

Why `with open()` instead of `open()/close()`? Show a case where the file stays open because of an exception.

Q425 ●●●○○○○○○○○ 3/10 [raise, exception chaining]

Write `parse_age(s)` raising `ValueError` with helpful messages for non-numeric, negative and above-150 values. Then catch and re-raise with context using `from`.

Q426 ●●●○○○○○○○○ 3/10 [custom exception hierarchy]

Design a hierarchy: `BankError`, `InsufficientFunds(balance, needed)` and `AccountClosed`. Catch the base class in one place and show why hierarchy helps.

Q427 ●●●○○○○○○○○ 3/10 [try/else]

What is the `else` clause of `try` for? Refactor a try block that wraps too much code.

Q428 ●●●○○○○○○○○ 3/10 [file modes]

Explain '`r`', '`w`', '`a`', '`r+`', '`x`', '`rb`'. Write a program that safely appends to a log and never overwrites.

Q429 ●●●○○○○○○○○ 3/10 [file statistics]

Count lines, words and the longest line of a file. Handle a missing file and an empty file gracefully.

Q430 ●●●○○○○○○○○ 3/10 [CSV by hand]

Read a CSV of students (`name,marks,city`), skip bad rows, print the top 3 and the average. Where do bad rows come from?

Q431 ●●●●○○○○○○ 4/10 [EAFP vs LBYL]

Rewrite five look-before-you-leap snippets in EAFP style and give one case where LBYL is better.

Q432 ●●●●○○○○○○ 4/10 [JSON round trips]

Save a nested dict/list to JSON and load it back. Which types do not survive a round trip (tuples, sets, int keys)? Write a workaround.

Q433 ●●●●○○○○○○ 4/10 [restoring state]

Write a `with`-helper using `try/finally` that temporarily changes the current directory or redirects `print` to a file and restores everything even on error.

Q434 ●●●●○○○○○○ 4/10 [chunked reading]

Process a huge file in chunks of N bytes or lines without loading it. Count something across chunk boundaries correctly (for example a word split between chunks).

Q435 ●●●●○○○○○○ 4/10 [config parsing]

Parse an INI-like file (`[section]`, `key=value`, `# comments`) into a nested dict and report line numbers in error messages.

Q436 ●●●●○○○○○○ 4/10 [binary files]

Write a list of ints as 4-byte big-endian numbers into a binary file and read it back using `int.to_bytes` and `int.from_bytes`.

Q437 ●●●●●○○○○○ 5/10 [robust input]

Simulate reading numbers from lines: skip blanks, allow commas and spaces as separators, report bad tokens with positions and continue.

Q438 ●●●●●○○○○○ 5/10 [atomic writes]

Update a file atomically: write to a temporary file and then rename it over the original. Why is this safer?

Q439 ●●●●●○○○○○ 5/10 [retry policy]

Write `read_with_retry(path, attempts)` that retries transient errors but NOT `FileNotFoundError`. How do you decide which exceptions to retry?

Q440 ●●●●●○○○○○ 5/10 [collecting errors]

Validate a form dict (name, age, email) and raise ONE error listing all problems instead of failing at the first.

Q441 ●●●●●○○○○○ 5/10 [report generation]

Given a log file with lines `LEVEL message`, produce a summary file. Malformed lines go to `errors.txt` with reasons.

Q442 ●●●●●○○○○○ 5/10 [\[tracebacks\]](#)

Create a 3-level deep error, catch it at the top and print only the innermost function name and line number using `e.__traceback__`.

Q443 ●●●●●○○○○○ 6/10 [\[persistence\]](#)

Build a persistent dict `put/get/delete` stored in a file that survives restarts. Make writes crash-safe (append-only log plus compaction).

Q444 ●●●●●○○○○○ 6/10 [\[tiny database\]](#)

Store records as JSON lines. Write functions to query by field, update and delete. How would an index speed up lookups?

Q445 ●●●●●○○○○○ 6/10 [\[file diff\]](#)

Compare two text files and print a unified-like diff using the longest common subsequence of lines.

Q446 ●●●●●○○○○○ 6/10 [\[encodings\]](#)

Read a file with non-ASCII text (accents, Devanagari, emoji) in UTF-8. Count characters versus bytes. What breaks with `encoding='ascii'`? Handle errors with `'replace'` and `'ignore'`.

Q447 ●●●●●○○○○○ 7/10 [\[undo systems\]](#)

Implement a text editor buffer with undo/redo, save and open. Ensure an exception during save never corrupts the original file.

Q448 ●●●●●○○○○○ 7/10 [\[validation frameworks\]](#)

Build a mini validation library: rules are functions raising `ValidationError(field, msg)`. Run all rules on records from a file and produce a report with counts by rule.

Q449 ●●●●●○○○○○ 8/10 [\[exceptions in generators\]](#)

What happens when an exception is raised inside a generator? What does `throw()` do? Write a generator that continues after handling a bad item.

Q450 ●●●●●○○○○○ 9/10 [\[robust design\]](#)

Design a robust "load CSV into memory" tool that handles a missing header, extra columns, embedded commas and newlines in quotes, duplicate keys and huge size. Write the parser and 15 test cases that break naive solutions.

CHAPTER 16

Grids & Matrices

2D problems ka ek hi darr hai: index galat ho jana. Direction arrays, bounds check aur visited set ko apni aadat banao.

Q451 ●●○○○○○○○○ 2/10 [nested lists, formatting]

Create an $n \times m$ matrix of zeros, one with row indices, and one with $i*j$. Print them with aligned columns.

Q452 ●●○○○○○○○○ 2/10 [sums]

Compute row sums, column sums, both diagonals and the sum of the border of a matrix.

Q453 ●●○○○○○○○○ 2/10 [traversal orders]

Print a matrix column by column, in a snake (zig-zag) order, and along its anti-diagonals.

Q454 ●●○○○○○○○○ 2/10 [transpose]

Transpose an $n \times m$ matrix in two ways, then in place for square matrices.

Q455 ●●●○○○○○○○ 3/10 [rotation]

Rotate a square matrix 90 degrees clockwise: with extra memory, with `zip`, and in place layer by layer.

Q456 ●●●○○○○○○○ 3/10 [searching in matrices]

Find all positions of a value, the maximum and its position, and whether some row or column is all equal.

Q457 ●●●○○○○○○○ 3/10 [neighbours]

Write `neighbors(r, c, R, C)` yielding the 4- and 8-neighbours that are in bounds using direction lists. Test at corners and on $1 \times N$ grids.

Q458 ●●●○○○○○○○ 3/10 [matrix properties]

Check whether a matrix is symmetric, a magic square, or Toeplitz (each diagonal is constant).

Q459 ●●●○○○○○○○ 3/10 [images as matrices]

Treat a 0/1 matrix as an image: flip horizontally, flip vertically, invert, and compute the bounding box of the 1s.

Q460 ●●●○○○○○○○ 3/10 [matrix multiplication]

Naive matrix multiplication, identity matrix, matrix power by repeated squaring. Use it to compute Fibonacci(1000) modulo 10^9+7 .

Q461 ●●●●○○○○○○ 4/10 [spiral]

Spiral order traversal of any $n \times m$ matrix, then generate an $n \times n$ spiral matrix.

Q462 ●●●●○○○○○ 4/10 [in-place marking]

If an element is 0, set its whole row and column to 0, in place with $O(1)$ extra space.

Q463 ●●●●○○○○○ 4/10 [connected regions]

Number of islands (4-connected) with iterative DFS. Then the largest island area, then the number of DISTINCT island shapes.

Q464 ●●●●○○○○○ 4/10 [cellular automata]

One step of Conway's Game of Life on a finite grid. Run a blinker and a glider and print each generation.

Q465 ●●●●○○○○○ 4/10 [Minesweeper counts]

Given mine positions, compute the numbers on every cell. Implement reveal with flood fill of zeros.

Q466 ●●●●○○○○○ 4/10 [grid DP]

Minimum path sum from top-left to bottom-right (right or down), and print the actual path.

Q467 ●●●●○○○○○ 4/10 [backtracking on grids]

Check whether a word exists in a letter grid by adjacent cells (no cell reuse) using backtracking.

Q468 ●●●●○○○○○ 5/10 [BFS shortest path]

Shortest path in a maze with walls from S to E and return the path. What changes if diagonal moves are allowed? If cells have costs?

Q469 ●●●●○○○○○ 5/10 [multi-source BFS]

Rotting oranges: minutes until all oranges rot, or -1 if impossible.

Q470 ●●●●○○○○○ 5/10 [validation]

Validate an n squared by n squared Sudoku grid for any n .

Q471 ●●●●○○○○○ 5/10 [2D prefix sums]

Sum of any submatrix in $O(1)$ after preprocessing. Answer 100,000 queries quickly.

Q472 ●●●●○○○○○ 5/10 [DP on matrices]

Largest square of 1s in a 0/1 matrix (DP). Then the largest rectangle (harder).

Q473 ●●●●○○○○○ 5/10 [symmetry checks]

Check whether one matrix is a rotation of another (0, 90, 180, 270 degrees) or a reflection.

Q474 ●●●●○○○○○ 6/10 [boundary flood fill]

Capture regions: flip all '0' regions completely surrounded by 'X'. Regions touching the border must stay.

Q475 ●●●●●○○○ 6/10 [constrained DP]

Count unique paths with obstacles, then with the constraint of at most 2 turns.

Q476 ●●●●●○○○ 6/10 [weighted shortest path]

Least-cost path where entering a cell costs its value. Implement it with a hand-written heap or `heapq`.

Q477 ●●●●●○○○ 7/10 [number-theory visualization]

Print an Ulam spiral of primes for $n = 21$ and mark the diagonals that are rich in primes.

Q478 ●●●●●○○○ 7/10 [histogram stack]

Largest rectangle of 1s in a binary matrix using the histogram-stack trick per row.

Q479 ●●●●●○○○ 8/10 [state space]

3x3 sliding tile puzzle: BFS to solve small shuffles and count the reachable states (it should be 181,440).

Q480 ●●●●●○○○ 9/10 [online exploration]

Robot cleaner: the grid is unknown and the robot only has local moves (`move`, `turn`, `clean`, wall sensor). Simulate a robot that provably covers every reachable cell using DFS with relative coordinates and backtracking moves.

Two Pointers, Windows, Prefix Sums & Greedy

Brute force $O(n^2)$ ko $O(n)$ me badalne ke 4 tareeke: two pointers, sliding window, prefix sums, greedy. Yahan wahi practice hai.

Q481 ●●○○○○○○○○ 2/10 [two pointers]

Reverse an array, check a palindrome, and remove duplicates from a sorted list in place using two indices.

Q482 ●●○○○○○○○○ 2/10 [prefix sums]

Build prefix sums and answer range-sum queries `[l, r]` in $O(1)$. What does it cost to update one element?

Q483 ●●○○○○○○○○ 2/10 [sorted two-sum]

Two-sum on a sorted list with two pointers. Then count pairs with sum less than the target.

Q484 ●●●○○○○○○○ 3/10 [fixed windows]

Maximum sum of a window of size k , and the minimum window length with sum at least S .

Q485 ●●●○○○○○○○ 3/10 [greedy]

Minimum coins with "nice" denominations greedily. Then find a set of coin denominations where greedy fails.

Q486 ●●●○○○○○○○ 3/10 [pointer merging]

Merge two sorted lists with pointers. Then merge k lists by repeatedly merging pairs and count the work.

Q487 ●●●○○○○○○○ 3/10 [partitioning]

Partition a list around a pivot (less than, equal to, greater than) in $O(1)$ extra space.

Q488 ●●●○○○○○○○ 3/10 [difference arrays]

Apply many range updates "add v to $[l, r]$ " efficiently using a difference array and print the final list.

Q489 ●●●●○○○○○○ 4/10 [sliding window]

Shortest subarray with sum at least S (positive numbers) with a sliding window. Why does the window trick need positive numbers?

Q490 ●●●●○○○○○○ 4/10 [window with dict]

Longest substring with at most 2 distinct characters, then with at most k .

Q491 ●●●●○○○○○○ 4/10 [k pointers]

3Sum closest to a target, then 4Sum with unique quadruplets.

Q492 ●●●●○○○○○ 4/10 [counting with sorting]

Count triplets in a list that can form a valid triangle. Sort first and explain why two pointers work.

Q493 ●●●●○○○○○ 4/10 [greedy reachability]

Jump game: can you reach the last index? Then the minimum number of jumps. Explain why greedy is correct.

Q494 ●●●●○○○○○ 4/10 [greedy, circular]

Gas station circuit: find the starting station that completes the circle. Prove that if total gas is at least total cost, a solution exists.

Q495 ●●●●○○○○○ 4/10 [sort plus greedy]

Assign cookies to children (greed factors versus sizes) to maximize satisfied children. Then: minimum arrows to burst balloons.

Q496 ●●●●○○○○○ 5/10 [prefix remainders]

Count the subarrays whose sum is divisible by k using prefix remainder counts. Be careful with negative numbers.

Q497 ●●●●○○○○○ 5/10 [products, negatives]

Maximum product subarray (negatives and zeros!). Track both the maximum and the minimum.

Q498 ●●●●○○○○○ 5/10 [fixed window, counts]

Find all start indices of anagrams of p in s using a fixed window and counts.

Q499 ●●●●○○○○○ 5/10 [permutation in string]

Does s contain a permutation of t as a substring? Then find the smallest window containing all characters of t .

Q500 ●●●●○○○○○ 5/10 [two passes]

Distribute the minimum candies so that higher-rated neighbours get more. Explain why two passes suffice.

Q501 ●●●●○○○○○ 5/10 [scheduling]

Tasks with a cooldown n between equal tasks: minimum time to finish all. Derive a formula, then simulate to verify it.

Q502 ●●●●○○○○○ 5/10 [monotonic stack]

Daily temperatures (days until a warmer day) and the largest rectangle in a histogram. State the stack invariant.

Q503 ●●●●○○○○○ 6/10 [single pass, $O(1)$ space]

Longest "mountain" subarray in one pass and $O(1)$ space.

Q504 ●●●●●○○○○ 6/10 [Kadane variants]

Maximum circular subarray sum, and maximum subarray sum with at most one deletion.

Q505 ●●●●●○○○○ 6/10 [amortized analysis]

"Fruit into baskets" (two types) as a sliding window. Generalize, then explain why it is $O(n)$ even though there is a `while` inside a `for`.

Q506 ●●●●●○○○○ 6/10 [interval problems]

Insert a new interval into a sorted non-overlapping list, remove covered intervals, and find the free gaps between meetings.

Q507 ●●●●●○○○ 7/10 [event ordering]

Minimum number of platforms (or meeting rooms) via sorted starts and ends with two pointers.

Q508 ●●●●●○○○ 7/10 [greedy with counts]

Reorganize a string so no two adjacent characters are equal, or report that it is impossible. Use counts and a greedy choice.

Q509 ●●●●●○○○ 8/10 [greedy proof, trees]

Huffman coding: build the tree from frequencies, print the codes, compress and decompress a sentence, and compare bit counts.

Q510 ●●●●●○○○ 9/10 [greedy proof by brute force]

Remove exactly k digits from a number string so the result is the smallest possible (no leading zeros). Use a monotonic stack, then prove it by brute force on all strings up to length 8.

CHAPTER 18

Stacks, Queues, Linked Structures & Heaps

Yeh data structures sirf list ke upar rules hain. Rule samjho, phir code khud likhna aasan ho jayega.

Q511 ●●○○○○○○○ 2/10 [stack with a list]

Use a list as a stack: reverse a string and check that brackets in '`{[{}]}`' are balanced. Report the position of the first error.

Q512 ●●○○○○○○○ 2/10 [queue performance]

Use a list as a queue and measure `pop(0)` for 100,000 items against `collections.deque.popleft()`. Explain the difference.

Q513 ●●○○○○○○○ 2/10 [two stacks]

Simulate browser history (back, forward, visit) with two stacks.

Q514 ●●●○○○○○○○ 3/10 [expression evaluation]

Evaluate a postfix expression '`3 4 + 2 *`'. Then convert infix to postfix with precedence and parentheses (shunting-yard).

Q515 ●●●○○○○○○○ 3/10 [augmented stack]

A stack with `push`, `pop` and `get_min`, all in $O(1)$.

Q516 ●●●○○○○○○○ 3/10 [amortized cost]

Implement a queue with two stacks and explain the amortized cost. Then a stack with two queues.

Q517 ●●●○○○○○○○ 3/10 [linked lists]

A singly linked list with small node classes: `append`, `prepend`, delete by value, reverse, find the middle, detect a cycle.

Q518 ●●●○○○○○○○ 3/10 [nesting validation]

Validate a string of nested tags like `<a><b></b></a>` and report the position of the first error.

Q519 ●●●○○○○○○○ 3/10 [stack reduction]

Remove adjacent duplicates repeatedly with a stack ('abbaca' gives 'ca'). Then with k duplicates ('deeedbbcccbdaa', k = 3).

Q520 ●●●●○○○○○ 4/10 [circular buffers]

A circular queue with fixed capacity: enqueue, dequeue, full/empty detection without a separate counter.

Q521 ●●●●○○○○○ 4/10 [iterative decoding]

Decode 2[ab]3[c] and nested versions using a stack instead of recursion.

Q522 ●●●●○○○○○ 4/10 [monotonic stacks]

Next greater element, previous smaller element and the stock span problem, all with a monotonic stack. Explain amortized $O(n)$.

Q523 ●●●●○○○○○ 4/10 [path processing]

Simplify a Unix path: '/a/./b/./../c/' becomes '/c'.

Q524 ●●●●○○○○○ 4/10 [linked list problems]

Merge two sorted linked lists, remove the n -th node from the end in one pass, and check whether a linked list is a palindrome.

Q525 ●●●●○○○○○ 4/10 [monotonic deque]

Sliding window maximum with a deque of indices. Why is each index pushed and popped at most once?

Q526 ●●●●○○○○○ 4/10 [priority handling]

Simulate a hospital emergency queue: patients (name, severity, arrival), most severe first, ties by arrival. Implement with sorted insertion and with a heap, then compare.

Q527 ●●●●○○○○○ 5/10 [heap by hand]

Implement a binary min-heap, then find the k largest items of a stream using a heap of size k .

Q528 ●●●●○○○○○ 5/10 [top-k]

Top- k frequent words with ties alphabetical. Implement with a heap and with bucket sort.

Q529 ●●●●○○○○○ 5/10 [k-way merge]

Merge k sorted lists with a heap. Count comparisons versus pairwise merging.

Q530 ●●●●○○○○○ 5/10 [cache design]

Implement an LFU cache (least frequently used, ties broken by least recent) with $O(1)$ operations.

Q531 ●●●●○○○○○ 5/10 [scheduling simulation]

Round-robin CPU scheduling with a time quantum: compute average waiting and turnaround times.

Q532 ●●●●○○○○○ 5/10 [calculator]

A basic calculator with + - * /, parentheses and unary minus, stack-based.

Q533 ●●●●○○○○○ 6/10 [heaps, streaming]

Running median with two heaps, then sliding window median.

Q534 ●●●●●○○○○ 6/10 [doubly linked lists]

A doubly linked list with sentinel nodes: $O(1)$ insert and delete given a node. Build a text cursor with left, right, insert and backspace.

Q535 ●●●●●○○○○ 6/10 [random pointers]

Copy a linked list where each node has an extra "random" pointer to any node or None: first with a dict of old to new, then with $O(1)$ extra space by interleaving.

Q536 ●●●●●○○○○ 6/10 [expression trees]

Build an expression tree from a postfix expression, evaluate it, and print it back in infix with minimal parentheses.

Q537 ●●●●●○○○○ 7/10 [stack-sortable permutations]

Decide whether a permutation can be sorted using a single stack (push and pop in order) and output the operations.

Q538 ●●●●●○○○○ 7/10 [linked list surgery]

Reorder a linked list $L_0, L_1, \dots, L_n$ into $L_0, L_n, L_1, L_{n-1}, \dots$. Then sort a linked list in $O(n \log n)$ with merge sort.

Q539 ●●●●●○○○○ 8/10 [system design]

Design a rate limiter: at most k requests per user per t seconds using a sliding-window log with dequeues. Test with adversarial timings and discuss memory.

Q540 ●●●●●○○○○ 9/10 [two-stack editor]

Implement a text editor with insert, delete, cursor moves and undo/redo of any number of steps, all in amortized $O(1)$ per operation, using two stacks.

Real-World Mini Projects & Data Wrangling

Ab tak tools seekhe, ab unhe jodo. Har question ek chhota software hai: pehle data model socho, phir code likho.

Q541 ●●●○○○○○○○ 3/10 [dicts, sorting, statistics]

Given students and marks in a dict, produce a report: rank list, grade distribution, average, median, and the top scorer per subject.

Q542 ●●●○○○○○○○ 3/10 [pricing rules, integers for money]

A shopping cart with items, quantities and discount rules (buy 3 pay 2, 10% off above 1000, coupon codes). Compute the total and an itemised bill. Avoid float bugs by using integer paise.

Q543 ●●●○○○○○○○ 3/10 [search, dict, uniqueness]

A contact book: add, search by partial name (case-insensitive), delete, list sorted, and detect duplicates by phone number.

Q544 ●●●○○○○○○○ 3/10 [random, string, guarantees]

Write a password strength scorer and a random password generator that always includes every character class (use `random.choice`).

Q545 ●●●○○○○○○○ 3/10 [lists, streaks, moving averages]

Given daily temperatures for a month: hottest and coldest days, longest warm streak, and a moving 7-day average.

Q546 ●●●○○○○○○○ 3/10 [dict, tiered rules]

Parking lot: slots, entry and exit times, fee calculation with tiered pricing (first hour, each next hour, daily cap). Reject entry when full.

Q547 ●●●●○○○○○○○ 4/10 [transactions, atomicity]

Inventory system with restocking rules, a low-stock report, and sale transactions that fail atomically if any item is short.

Q548 ●●●●○○○○○○○ 4/10 [parsing, streaks]

From lines `'date,student,P/A'` compute the attendance percentage per student, the students below 75%, and each student's longest absent streak.

Q549 ●●●●○○○○○○○ 4/10 [algorithm design, duplicates]

Wordle checker: given a secret and a guess produce green/yellow/gray feedback, handling repeated letters correctly. Then filter a word list to those consistent with all feedback so far.

Q550 ●●●●○○○○○ 4/10 [aggregation, outliers]

Parse bank transactions (`date`, `description`, `amount`). Compute monthly totals, the largest expense category, and flag anything above 3 times its category average.

Q551 ●●●●○○○○○ 4/10 [game AI, simulation]

Tic-tac-toe where the computer wins if possible, else blocks, else takes the centre, else a corner. Play it against a random player 1000 times and report the results.

Q552 ●●●●○○○○○ 4/10 [state, input validation]

Hangman logic: masked word, guessed letters, remaining attempts. Handle repeated guesses and invalid input.

Q553 ●●●●○○○○○ 4/10 [calendar arithmetic]

Without `datetime`: days in a month, leap years, `add_days(date, n)`, `days_between(d1, d2)` and `weekday(date)`. Verify against 20 known dates.

Q554 ●●●●○○○○○ 5/10 [text analytics]

Given a text, compute readability statistics (sentences, words, an estimate of syllables), the top bigrams, and a frequency histogram printed with `#`.

Q555 ●●●●○○○○○ 5/10 [inverted index]

Build an inverted index over documents. Support AND, OR and NOT queries and rank by term count.

Q556 ●●●●○○○○○ 5/10 [similarity, recommendation]

Given user to set of liked movies, recommend movies to a user using Jaccard similarity with other users.

Q557 ●●●●○○○○○ 5/10 [dependencies, critical path]

Given tasks with durations, deadlines and dependencies, produce a valid schedule or explain why it is impossible, and report the critical path.

Q558 ●●●●○○○○○ 5/10 [simulation, assignment]

Simulate two elevators and a list of requests. Assign each request to the nearest idle elevator and report the total waiting time.

Q559 ●●●●○○○○○ 5/10 [exact arithmetic]

Split a bill among `n` people using integers (paise) so the parts sum exactly to the total. Distribute the remainder fairly and deterministically.

Q560 ●●●●○○○○○ 5/10 [dependency evaluation]

A mini spreadsheet: cells hold numbers or formulas like `'=A1+B2*2'`. Evaluate in dependency order and detect circular references.

Q561 ●●●●●○○○○ 5/10 [grid game]

Battleship: validate a board with ships of given lengths (no touching, no overlap), then process shots and report hit, miss, sunk and win.

Q562 ●●●●●○○○○ 6/10 [prefix search, tries]

Autocomplete: given a dictionary of words with frequencies and a prefix, return the top 3. Build it first with sorting and `bisect` by hand, then with a Trie class.

Q563 ●●●●●○○○○ 6/10 [time-based design]

A cache with TTL: `set(k, v, ttl)` and `get(k)` with lazy expiry. Simulate time with a clock function passed in.

Q564 ●●●●●○○○○ 6/10 [classification logic]

A poker hand ranker for 5 cards: straight flush, four of a kind, full house and so on. Compare two hands.

Q565 ●●●●●○○○○ 6/10 [invariants, ledgers]

A double-entry ledger where every transaction has equal debits and credits. Verify the books balance, produce account balances, and reject unbalanced transactions.

Q566 ●●●●●○○○○ 6/10 [permutation search]

Delivery routes: given a road graph with distances, find the shortest route visiting 4 places in any order by brute force permutations you write yourself.

Q567 ●●●●●○○○ 7/10 [parsing, safe evaluation]

A small rules engine: rules such as `age > 18 and country in ['IN', 'US']` stored as strings. Parse them safely (no `eval`) and evaluate on dict records.

Q568 ●●●●●○○○ 7/10 [edit scripts]

Given two lists of strings compute a minimal edit script and apply it to the first list to obtain the second.

Q569 ●●●●●○○○ 8/10 [Monte Carlo]

Bank simulation with `k` tellers and random arrivals (`random.seed` for repeatability). Estimate the average waiting time for `k = 1, 2, 3` and explain the results.

Q570 ●●●●●○○○ 9/10 [capstone]

Build a command-line to-do engine (no UI): add, edit, delete, tags, due dates, priorities, search and filter, undo, and save/load JSON. Design the data model first, then write 20 assert-based tests.

Debugging, Output Prediction & Python Puzzles

Asli master wo hai jo code chalaye bina bata sake ki kya hoga. Har question: pehle predict karo, phir run karo, phir explain karo.

Q571 ●●○○○○○○○○ 2/10 [strings, sets, counting]

Predict without running: `print(len(set("mississippi")), "mississippi".count("ss"), "mississippi".find("ss", 3)).`

Q572 ●●○○○○○○○○ 2/10 [off-by-one bugs]

Both functions are meant to work as named. Find and fix the bugs, then write two tests for each:

```
def last_n(lst, n):          # last n items
    return lst[len(lst)-n-1:]

def count_up(n):             # print 1..n
    i = 0
    while i < n:
        print(i)
        i += 1
    print(n)
```

Q573 ●●○○○○○○○○ 2/10 [comprehension scope]

Predict what `print([i for i in range(3)], i)` does in Python 3 and explain why.

Q574 ●●○○○○○○○○ 2/10 [range behaviour]

Predict: `list(range(5,0)), list(range(5,0,-1)), range(0,10,3)[-1], len(range(2,20,5)), 5 in range(0,10,5), range(3) == range(0,3).`

Q575 ●●●○○○○○○○○ 3/10 [print internals]

Predict the output: `print(1, 2, sep='-', end='!'), print('a' 'b'), print('a', 'b'*2, 3*'c'), print(*'abc'), print(sep='')`.

Q576 ●●●○○○○○○○○ 3/10 [slice assignment]

Predict the final list: `a = [1,2,3]; a[5:] = [9]; a[1:2] = []; a[::2] = ['x','y']`. Explain each step.

Q577 ●●●○○○○○○○○ 3/10 [strings are immutable]

Why does `for c in 'hello': c.upper()` change nothing? Why does `s.replace('l', 'L')` not change `s`? List five other string methods that return new strings.

Q578 ●●●○○○○○○○○ 3/10 [debugging technique]

Use print debugging and `assert` to find the bug in a binary search that loops forever on `([1,2,3,4], 4)`. Write the failing test first.

Q579 ●●●○○○○○○○ 3/10 [scope]

Predict the output:

```
x = 'global'
def f():
    x = 'local'
    def g():
        print(x)
    return g
f()()
```

Now change it so that `g` modifies the `x` of `f`.

Q580 ●●●●○○○○○○○ 4/10 [gotchas]

For each expression predict the output and explain: `[1,2,3] == [1,2,3]`, `[1,2,3] is [1,2,3]`, `'a' in 'abc'`, `'' in 'abc'`, `[] in [[]]`, `() == []`, `not not []`, `bool('False')`.

Q581 ●●●●○○○○○○○ 4/10 [methods returning None]

Does `sorted([3,1,2], reverse=True) == [3,1,2].sort(reverse=True)`? Explain. List all list, dict and set methods that mutate and return `None`.

Q582 ●●●●○○○○○○○ 4/10 [loop variables]

After `for i in range(5): pass`, what is `i`? After `for i in []: pass`? After a comprehension? Describe a real bug this could cause.

Q583 ●●●●○○○○○○○ 4/10 [dict mutation]

```
d = {'a': 1, 'b': 2}
for k in d:
    d[k + 'x'] = 0
```

Predict what happens and fix it. What about changing values (not keys) while iterating?

Q584 ●●●●○○○○○○○ 4/10 [numeric surprises]

Explain: `0.1*3`, `round(2.5)`, `int(-3.7)`, `-7 // 2`, `-7 % 3`, `7 % -3`, `divmod(-7, 2)`, `2**-1`, `10**20 == 1e20`, `int(1e20) == 10**20`.

Q585 ●●●●●○○○○○○○ 5/10 [unpacking errors]

Predict each result or error: `a, b = 'xy'`, `a, b = {1:2, 3:4}`, `a, *b = 'x'`, `[a, b] = (1, 2)`, `(a, b), c = (1, 2), 3`, `a, b = 1, 2, 3`.

Q586 ●●●●●○○○○○○○ 5/10 [sort stability]

Predict `sorted(['bb', 'a', 'cc', 'd'], key=len)` and the same with `reverse=True`. Are ties kept in original order in the reversed sort? Explain.

Q587 ●●●●●○○○○○ 5/10 [shallow copies]

Predict the three printed lists:

```
a = [[1, 2], [3, 4]]
b = a[:]
c = [row[:] for row in a]
b[0][0] = 99
print(a, b, c)
```

Q588 ●●●●●○○○○○ 5/10 [class attributes]

```
class A:
    x = []
    def add(self, v):
        self.x.append(v)
a, b = A(), A()
a.add(1); b.add(2)
print(a.x, b.x, A.x)
```

Predict the output and fix it so each instance has its own list.

Q589 ●●●●●○○○○○ 5/10 [exhausted generators]

Predict `g = (x*2 for x in range(3)); print(list(g), list(g))`, then `sum(g)`. Explain what "consumed" means.

Q590 ●●●●●○○○○○ 5/10 [split behaviour]

Predict: `'a,b,,c'.split(',')`, `'a b'.split(' ')`, `' a b '.split()`, `'a-b-c'.rsplit('-', 1)`, `''.split(',')`, `''.split()`. What does `'abc'.split('')` do?

Q591 ●●●●●○○○○○ 5/10 [hash equality]

Predict `{1: 'a', 1.0: 'b', True: 'c'}` and its length. Why? What about `{[]: 1}` and `{(1, []): 1}`?

Q592 ●●●●●○○○○○ 6/10 [local variable trap]

Predict what happens:

```
total = 0
def add(n):
    total += n
    return total
print(add(5))
```

Fix it in three different ways and choose the best one.

Q593 ●●●●●○○○○○ 6/10 [string comparison]

Predict: `'abc' < 'abd'`, `'Z' < 'a'`, `'10' < '9'`, `'apple' < 'Apple'`, `sorted(['b', 'A', 'a', 'B'])`, `len('ß'.upper())`. What rule explains them?

Q594 ●●●●●○○○○ 6/10 [try/finally interplay]

Predict the printed output and return value:

```
def f():
    try:
        return 'try'
    finally:
        print('finally')
    return 'after'
print(f())
```

Now put `return 'finally'` inside the `finally` block. Then a version where an exception in `try` is swallowed by a `return` in `finally`. Explain the danger.

Q595 ●●●●●○○○○ 7/10 [mutating during iteration]

Predict the output:

```
a = [1, 2, 3]
for x in a:
    if x == 2:
        a.remove(x)
    print(x)
```

Then explain why an element gets skipped, using the iterator's internal index.

Q596 ●●●●●○○○○ 7/10 [debug by testing]

This function should return all primes up to and including `n` but gives wrong results for some inputs. Find the bugs BY TESTING, not by reading:

```
def primes(n):
    result = []
    for i in range(2, n):
        for j in range(2, int(i**0.5)):
            if i % j == 0:
                break
        else:
            result.append(i)
    return result
```

Q597 ●●●●●○○○○ 8/10 [testing methodology]

Pick a "clever" solution that is subtly wrong (for example Kadane's algorithm that resets at 0 and fails on all-negative arrays). Write a brute-force checker and a random test generator (`random.seed`) to find a counterexample. Explain how you shrink the failing input.

Q598 ●●●●●○○○○ 8/10 [performance intuition]

For each pair predict which is faster and by how much, then measure: `x in list` vs `x in set`; `s += c` vs `join`; `list.insert(0, x)` vs `append`; `sorted` on sorted vs random data; `a + b` vs `a.extend(b)` on big lists.

Q599 ●●●●●○○○○ 9/10 [puzzles]

Write a quine (a program that prints its own source code, without reading files). Then a function whose output differs each time it is called but does not use randomness.

Q600 ●●●●●●●●○ 9/10 [\[personal bug list\]](#)

Take your five longest programs from earlier chapters. For each, write five nasty inputs (empty, single element, duplicates, huge, negative) and make them all pass. Record every bug you find. This becomes your personal bug list.

Trigger words to tools

Use only AFTER you have attempted a problem. Goal: see the words, name the tool in one second.

If the problem says...	Think...
count, frequency, how many of each	dict of counts
have I seen this before? unique? duplicate?	set
group by / bucket	dict of lists (key = what makes items 'equal')
pair or triple with a given sum	set/dict, or sort + two pointers
substring or subarray with a property	sliding window
sum of a range, many queries	prefix sums
many range updates	difference array
next greater / nearest smaller	monotonic stack
matching, nesting, undo	stack
shortest path / fewest steps	BFS (queue + visited)
all arrangements / all subsets / puzzles	recursion + backtracking
count ways / min cost, repeated subproblems	memoization then DP
sorted input, or 'minimum X such that...'	binary search
top k, running best, scheduling	heap
overlapping intervals	sort by start, sweep
dependencies, ordering	graph + topological sort
stream, huge data	generator, one pass, O(1) state

Progress tracker

Write the date or tick in each box. 'Clean' = solved alone. 'Redo' = needed a lookup, revisit at Day+3, +10, +30.

#	Chapter	Started	Clean	Redo done	Finished
1	Names, Objects & Types				
2	Numbers & Math Thinking				
3	Strings I: Slicing, Scanning & Classic Problems				
4	Strings II: Parsing, Formatting & Text Processing				
5	Lists: Manipulation, Patterns & Techniques				
6	Tuples, Unpacking & Data Modelling				
7	Dictionaries: Counting, Grouping, Lookup & Caching				
8	Sets: Uniqueness, Membership & Set Thinking				
9	Loops, Conditions & Simulation				
10	Functions: Design, Scope, Closures & Decorators				
11	Comprehensions, Generators & the Iteration Protocol				
12	Sorting & Searching: Keys, Stability & Binary Search Thinking				
13	Recursion & Backtracking				
14	Classes & Object-Oriented Design				
15	Exceptions & File Handling				
16	Grids & Matrices				
17	Two Pointers, Windows, Prefix Sums & Greedy				
18	Stacks, Queues, Linked Structures & Heaps				
19	Real-World Mini Projects & Data Wrangling				
20	Debugging, Output Prediction & Python Puzzles				